Jio Platforms Limited

HIGH-LEVEL SYSTEM ARCHITECTURE DOCUMENT

Consent Management System

COPYRIGHT INFORMATION
No part of this document may be copied, reproduced, stored in any retrieval system, or transmitted in any form or by any means, either electronically, mechanically, or otherwise, without prior written permission.

© Copyright Jio Platforms Limited.
All rights reserved.
Jio Platforms Limited, Reliance Corporate Park
Thane Belapur Road, Ghansoli-400 701

TRADEMARKS
All the brand names and other products or services mentioned in this document are identified by the trademarks or service marks of their respective owners

DISCLAIMER
The information in this document is subject to change without notice and should not be construed as a commitment by Jio Platforms Limited. assumes no responsibility or makes no warranties for any errors that may appear in this document and disclaims any implied warranty of merchantability or fitness for a particular purpose

"CONFIDENTIAL – FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Version History

Date	Contributor	Contribution	Ver	State
Jun 14, 2025	Shreay Kumar	Initial Draft	0.1	Draft
Jun 17, 2025	Mayur Jalora	Updated 1) Appendix: Consent Management System – Notifications Architecture 2) dated geolocation rules and removed point 4 from cookie consent service. 3) updated consent grievance service with adding consent id in grievance flow.	0.2	Draft
Jun 20, 2025	Lokesh Sharma	Reviewed and released document for phase 1 of NeGD Code for Consent challenge	1.0	Released
Nov 11, 2025	Mayur Jalora	Updated the Crypto module and overall document	1.1	Draft
Nov 20, 2025	Lokesh Sharma	Reviewed and released document for phase 2 of NeGD Code for Consent challenge	2.0	Released

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

References

Sr. No	Document Name and URL	Ver #
1	Business Requirement Document For Consent Management Under the DPDP Act, 2023	1.0
2	WCAG Accessibility Guidelines	2.1
3	OAuth 2.0 / OpenID Specifications	2.0

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Table of Contents

Version History *1*

References *2*

Table of Contents *3*

Table of Figures *6*

1. Introduction *7*

2. Document Overview *8*

2.1 Scope 8

2.2 Out of Scope 8

2.3 Assumptions 9

2.4 Document Structure 11

3. System Context *12*

3.1 Stakeholders and Their Roles 12

3.2 External Systems 12

3.3 Interfaces 13

3.4 Dataflows 13

4. Architecture Components *19*

4.1 Embedded UI Journeys (Served by CMS, Embedded by DF) 19

4.2 Core Domain Services 20

4.3 Cross-Cutting CMS Platform Services 20

4.4 Consent Service 22

4.5 Cookie Consent Service 23

4.6 Grievance Management Service 24

5. Consent Artifact — Immutability & Crypto Architecture *27*

5.1 System component participated 27

5.2 Detailed step-by-step procedure (mapping to system components) 28

5.3 Record & Index design (DB) 31

5.4 Security & Operational controls 31

5.5 Compliance mapping (summary) 31

6. Integration Interfaces *31*

6.1 Interface Categories and Actors 32

6.2 Embedded User Interfaces (Served by CMS) 32

6.3 System Integrations (Internal Service Interfaces) 32

6.4 Security & Compliance Considerations 33

6.5 Interface Usage Matrix (Summary) 33

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

7. Security Architecture 34
7.1 Authentication 34
7.2 Authorization 34
7.3 Data Protection 35
7.4 Audit & Logging 35
7.5 Security in Notifications 35
8. Data Architecture 36
8.1 Domain-Oriented Design 36
8.2 Storage Strategy & Principles 36
8.3 Data Retention and Lifecycle Policies 37
8.4 Multi-Tenant Isolation 37
8.5 Governance and Compliance 37
9. Deployment Architecture 38
9.1 Deployment Environments 38
9.2 Containerized Microservice Deployment 38
9.3 Scaling Strategy 39
9.4 Failover & High Availability 39
9.5 Blue-Green Deployment Strategy 39
10. Compliance & Governance 41
Appendix A: Glossary 43
Appendix B: JWS Signed Consent Artifact – Technical Specification 45
1) Purpose and Security Rationale 45
2) Structure of Consent Artifact (JWS Payload) 45
3) Sample Consent Payload 45
4) Signature & Headers 46
Appendix C: Authentication – Identifying the Data Principal 47
5) Auth 2.0 Flow 47
6) OpenID Connect (OIDC) Integration 47
7) Identity in JWT Consent Artifact 47
8) Operational Flow Summary 47
Appendix D: Notifications Service Architecture 48
High-Level Component 49
1) DynamicMockController 49
2) NotificationService 50
3) NotificationForwarderService 50

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

4) MongoDB Storage Layer 51

5) Configuration Architecture 51

Key config structures and design characteristics 51

Configuration Classes 51

6) Data Model (NotificationLog) 52

callbackResponse Structure 52

Expected `callbackRequest` Fields (from Consumer) 53

7) Runtime Flow Architecture 53

a) Callback (POST) Sequence 53

b) Forwarding (PUT) Sequence 53

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Table of Figures

Figure 1: Layered Architecture Diagram 12
Figure 2: Consent Management System Dataflow 14
Figure 3: Minor consent collection journey 15
Figure 4: Architectural Components in Consent Management System 19
Figure 5: Consent Service Architecture 22
Figure 6: Cookie Consent Management Service 23
Figure 7: Grievance and Tracking Service Architecture 25
Figure 8: Immutability & Crypto Architecture 27
Figure 9: Mapping to system components 28
Figure 10: Interfaces Integrations 32
Figure 11: Security Architecture 34
Figure 12: Data Architecture 36
Figure 13: Deployment Diagram 38
Figure 14: Blue Green Deployment Architecture 40
Figure 15: CMS Notification Architecture 48
Figure 16: CMS Notification sequence diagram (Internal) 49
Figure 17: Integration with Notification System 54

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

1. Introduction

The Consent Management System (CMS) is a critical component developed to ensure compliance with the Digital Personal Data Protection (DPDP) Act, 2023. As data privacy regulations become increasingly stringent, organisations are mandated to manage user consent in a transparent, secure, and auditable manner. The CMS addresses this need by offering an end-to-end framework for managing consent across its entire lifecycle—including collection, validation, updates, renewal, withdrawal, and auditing.

This system empowers Data Principals (users) with full control over their data while enabling Data Fiduciaries and Processors to handle data responsibly and under legal requirements. The CMS facilitates seamless integration with enterprise platforms, enforces purpose-specific consent protocols, and ensures real-time synchronization and compliance validation.

From a high-level architectural perspective, the CMS is designed to be modular, scalable, and API-driven, supporting multilingual interfaces, secure communication protocols, and robust audit logging mechanisms. It provides administrative capabilities such as role-based access control, data retention policy configuration, and grievance redressal workflows to maintain operational efficiency and legal compliance.

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

2. Document Overview

This document outlines the high-level architectural components of the CMS, including system interfaces, integration points, core modules, data flow design, and security posture to ensure that all functional and non-functional requirements of the Business Requirement Document [1] are met with technical robustness and scalability.

2.1 Scope

The High-Level System Architecture for the Consent Management System (CMS) will cover the following:

1) Consent Lifecycle Management
   - Collection, validation, updation, renewal, and withdrawal of consent.
   - Creation and secure storage of consent artifacts.
   - Real-time synchronization of consent status with internal and third-party systems (Data Fiduciary and Processors).

2) User Dashboard
   - Viewing of consent history with filtering and export options.
   - Capability to modify or revoke consent at a granular level.
   - Raise grievances and data access/correction/erasure requests.

3) Cookie Consent Management
   - Display of cookie notices with granular controls.
   - Real-time logging and updates of cookie preferences.
   - Auto-expiry and compliance with multi-lingual and accessibility requirements.

4) Notification System
   - Real-time user notifications (e.g., approvals, expirations, updates).
   - Alerts to Data Fiduciaries and Processors on consent changes via secure APIs.

5) Grievance Redressal Mechanism
   - Complaint logging, categorization, and tracking with escalation workflows.
   - Status notifications and resolution dashboards.

6) System Administration
   - Role-based access control (RBAC), and multi-factor authentication (MFA).
   - Configuration of data retention policies with automated purging and exemptions.

7) Logging and Audit
   - Immutable audit trails for all consent-related activities.
   - Support for compliance reporting and tamper-proof record maintenance.

2.2 Out of Scope

The following elements are considered out of scope for this system architecture:

"CONFIDENTIAL – FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

1) Third-Party Consent Frameworks: Integration with external platforms not compliant with the DPDP Act or outside the defined API interfaces.
2) Non-Personal or Anonymized Data Processing: Data usage scenarios that do not require explicit consent under the DPDP Act.
3) Legal/Regulatory Interpretation: Determining the legality of consent scenarios beyond technical enforcement (to be handled by legal/compliance teams).
4) Enterprise-Wide Identity Management: The CMS will integrate with existing Identity and Access Management (IAM) systems but not replace or manage core IAM functionality.
5) Data Processing Workflows: Actual data processing activities based on consent are managed by the Data Fiduciary systems; the CMS only governs consent status and audit.
6) Multi-jurisdictional Compliance: The system is designed specifically for the DPDP Act, 2023 (India). Other global privacy laws (e.g., GDPR, CCPA) are out of scope unless explicitly extended in future phases.

2.3 Assumptions

The architecture presented in this document is based on a set of assumptions derived from input received for this coding challenge and on expert opinion.

1) Identity of the Data Principal and Authentication of the Data Principal

Identity is a key element of any consent ecosystem. It is not feasible to give an individual control over their data (including control over access to this data), if that individual cannot be securely identified so the data can be determined to be theirs and securely authenticated to capture their intent in controlling that data. Therefore, consent scenario like the cookie consent, where the data principal is not necessarily identified, cannot yield consent data visible in a dashboard as the cookies are associated with a given device and not the user. Aadhaar, the dominant way of identifying citizens of India, is legally limited in terms of use. It is thus not feasible to assume, that it can be used to identify all data principals across all data processors. Neither can the existence of a universally applicable authentication method be assumed. Furthermore, the data processors (DP) and data fiduciaries (DF) must use the same identity of a data principal as the consent management system (CMS) to be able to link data use to a specific consent. Therefore, the system described below is assumed to use the data fiduciaries for authentication of the Data Principal.

2) Supervision/ Monitoring of Data Processing Activities

Full control over data requires full control of the system, including full control of all software and hardware used. A cloud-based object storage might not physically remove the data from all the hard drives involved as its deletion is requested, a malicious hosting operator could create additional copies of hard drives or memories of the systems in their custody, a misconfigured log file could allow the software to retain more data, than documented and a backup tape will contain data long after it has been removed from the live system. Therefore, full control over if the data processor has followed the consent to the letter requires full control over the systems of the data processor (DP). This is not assumed to be feasible as data processors are independent legal entities. Execution of the data processing based on consent, therefore, requires trusting the Data Processor. We, therefore, take the approach that supervisory activities such as periodic audits and reporting are sufficient to trust the data processor, and the role of the consent management system is to provide facilities for the data processor to act lawfully rather than attempt to enforce legal behavior by technical means.

3) No Central Consent Platform or Repository

"CONFIDENTIAL – FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Any architecture developed must take into account the unique size and structure of India. Although a centralized solution would be technologically possible, various similar solutions in India have successfully utilized a distributed architecture, minimizing scaling and availability challenges posed by a central solution. Such central solutions (like UIDAI and GSTN) have been implemented by Authorities with the building of an industry ecosystem around that solution for last-mile integrations. Such centralized platforms have specific objectives (Identity, Indirect Taxation) and a clear scope of responsibility of the Authority.

It is assumed that there is no such central infrastructure/ platform, hence the data principal will not have an aggregated view of the consents provided by him/her to the data fiduciaries(DFs).

Summary of Assumptions

- The Identity and Authentication of the Data Principal is the responsibility of the Data Fiduciary. The same Identity post-authentication will be passed by the data fiduciary (DF) to data processors (DP) and the consent management system (CMS)
- The adherence to the consent scope by the data processor (DP) is the responsibility of the data fiduciary (DF).
- The consent Management System (CMS) will have consents from the data fiduciary(DF) using this platform. It cannot show any consent which are not given by the Data Principal on the platform.

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

2.4 Document Structure

The document is structured as follows:

1) Introduction – Purpose and context of the system.
2) Document Overview – (This section).
3) System Context – Overview of stakeholders, external systems& interfaces, and data flows.
4) Architectural Components – Detailed view of core modules and services.
5) Integration Interfaces – APIs and communication protocols with third-party systems.
6) Security Architecture – Authentication, authorization, data protection, and audit.
7) Data Architecture – Data models, storage strategy, and retention policies.
8) Deployment Architecture – Environments, scaling strategy, and failover design.
9) Compliance & Governance – Measures to ensure compliance with the DPDP Act.
10) Appendices – Glossary and supporting diagrams.

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

3. System Context

The Consent Management System (CMS) operates within a data ecosystem that involves multiple stakeholders, external services, and regulatory requirements under the DPDP Act, of 2023. The system ensures transparent and legally compliant purpose-based consent processing, cookie consent management and grievance handling. The following image depicts the complete system context in a layered architecture design.

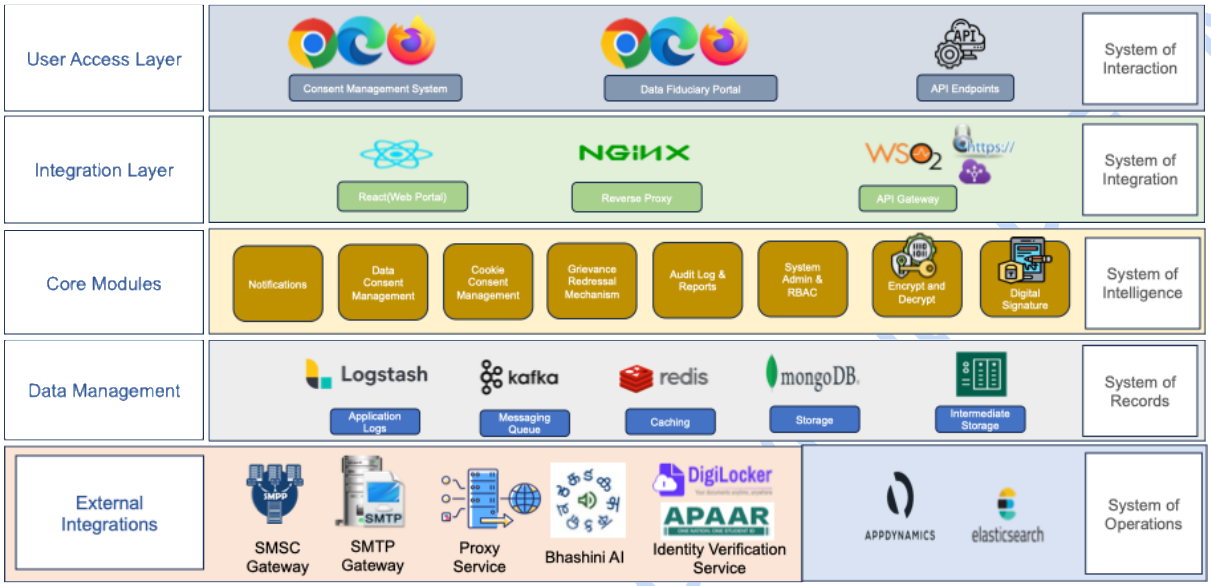

Figure 1: Layered Architecture Diagram

3.1 Stakeholders and Their Roles

Stakeholder	Role Description
Data Principal	The individual whose personal data is collected. Has rights to grant, update, or revoke consent.
Data Fiduciary (DF)	Determines the purpose and means of processing personal data. Must obtain and manage consent.
Data Processor (DP)	Processes data on behalf of the Data Fiduciary. Acts per instructions and contractually bound.
Consent Management System (CMS)	Core system managing the lifecycle of consent. Facilitates user interaction, stores artifacts, triggers notifications, and enforces compliance policies.
Data Protection Officer (DPO)	Oversees compliance, grievance redressal, and resolution tracking. Uses CMS to monitor actions and generate reports.
System Administrators	Manage access controls, data retention policies, user roles, and ensure audit readiness.

3.2 External Systems

1) **Identity and Verification Services:** e.g., DigiLocker for parent/guardian verification.
2) **Notification Services:** Email, and SMS gateways for sending user and fiduciary alerts.
3) **Third-Party Data Processors:** Integrated platforms that act on consented data (e.g., CRMs, analytics platforms).

"CONFIDENTIAL - FOR NeGD USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

4) **Third-Party Multilingual Service:** Multilingual Capability providers like Bhashini AI.

3.3 Interfaces

1) **Web/Mobile User Interfaces**: Used by Data Principals to interact with the CMS (e.g., providing or updating consent, raising grievances).
2) **Data Fiduciary Systems**: Business platforms integrated via APIs that initiate consent requests and validate status before processing personal data.
3) **Third-Party Data Processors**: External service providers (e.g., marketing, analytics) that access user data based on consent status shared by the CMS.
4) **Notification Services (Email/SMS/In-App)**: Used by the CMS to send real-time consent updates, alerts, and acknowledgments to users and system actors.
5) **Grievance and Ticketing Systems**: Integrated modules or services for complaint logging, tracking, and escalation.
6) **Identity Verification Services (e.g., DigiLocker)**: Used optionally to verify guardian identity for minor Data Principals.
7) **Audit and Compliance Dashboards**: Tools used by auditors and DPOs to monitor logs, generate reports, and track consent activities.

3.4 Dataflows

The system architecture ensures that while the data principal interacts primarily through the CMS web page embedded within the data fiduciary's (DF) platform, all consent-related processing—including collection, validation, updation, renewal, withdrawal, cookie preferences and grievances—is managed and controlled exclusively by the CMS.

The flow begins when a data principal accesses a service hosted by a data fiduciary. The DF, rather than collecting consent directly, redirects the user to the CMS interface embedded within their own platform. Here, the data principal provides purpose-specific consent, including granular options and explicit actions, which are stored immutably by the CMS along with metadata such as timestamp, purpose, language preference, and consent status. The CMS notifies the DF of consent completion via secure APIs. All the possible flows have been shown in the data flow diagram.

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

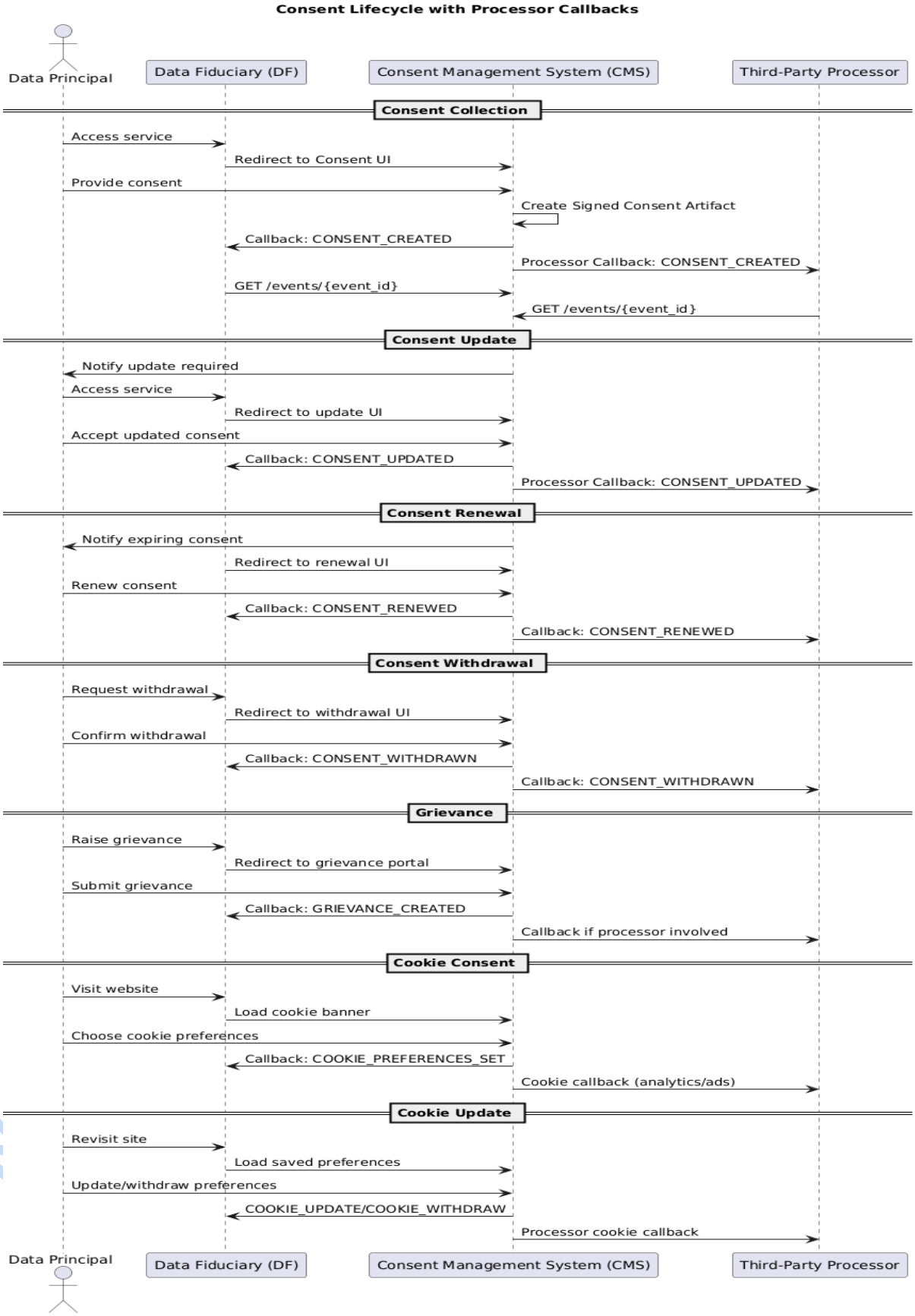

Figure 2: Consent Management System Dataflow

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

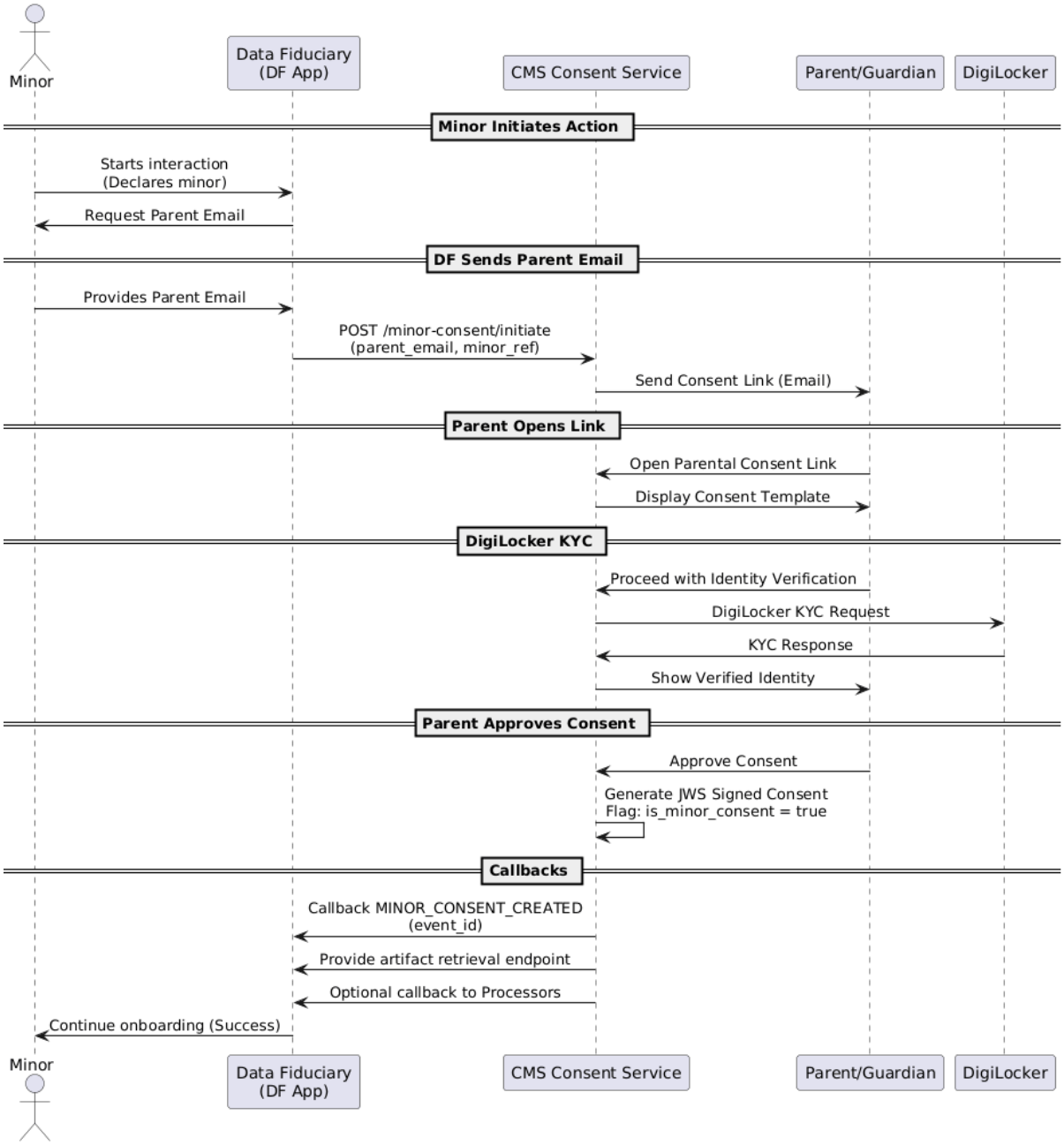

Figure 3: Minor consent collection journey

1) **Consent Collection (Standard Journey)**

- DF backend invokes CMS Consent UI for the Data Principal.
- Data Principal reviews:
  - Purpose(s)
  - Processing Activities
  - Data Items
  - Processors / Sharing Information
- Data Principal grants consent.
- CMS generates:
  - JWS-signed Consent Artifact
  - Version record
  - Complete audit entry

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- CMS sends callbacks to:
  - Data Fiduciary (DF)
  - Data Processor(s)
- CMS stores logs in immutable audit store.

---

2) Consent Validation

- DF calls CMS validation API.
- CMS validates:
  - JWS signature
  - Consent status
  - Purpose & processing rules
- CMS returns validation result.

---

3) Consent Update (DF-initiated)

- DF updates purpose/processing rules.
- CMS decides:
  - Material change → re-consent required
  - Minor change → silent version update
- CMS notifies Data Principal (if required).
- Data Principal accepts or rejects re-consent.
- CMS generates updated version under same Consent ID.
- DF & Processor receive CONSENT_UPDATED.

---

4) Consent Renewal

- CMS detects upcoming expiry.
- Sends notifications to:
  - Data Principal
  - DF
- User renews through Consent UI.
- CMS generates new version.
- Callback sent to DF & Processor: CONSENT_RENEWED.

---

5) Consent Withdrawal

- Data Principal withdraws consent.
- CMS sets status to Withdrawn.
- DF & Processor receive withdrawal callback.
- DF halts processing as per policy.

---

6) Grievance Handling

- Data Principal submits grievance via CMS UI.

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- CMS:
  - Creates ticket
  - Generates event ID
  - Sends callback to DF
- DF updates workflow status:
  - In-Progress
  - Resolved
  - Escalated
- CMS notifies Data Principal of updates.

---

7) Cookie Consent

- CMS cookie banner loads on DF app/website.
- Data Principal selects categories:
  - Essential
  - Functional
  - Analytics
  - Marketing
- CMS stores preferences and enforces them.

---

8) Cookie Preference Update

- Data Principal revisits cookie settings.
- Updates or withdraws categories.
- CMS logs updates and syncs enforcement.

---

9) Data Processor Callbacks (All Flows)

- CMS sends callbacks for:
- Consent Created
- Consent Updated
- Consent Withdrawn
- Consent Renewed
- Minor Consent Created
- Grievance Events
- Each callback includes:
  - event_id
  - resource_type
  - timestamp
  - minimal payload
  - signed headers for verification

---

10) **Parental Consent Collection**

A) Minor Interacts With DF

- The minor begins a service request on DF app/website.
- DF identifies the user as a minor (via declaration or DF policy).

"CONFIDENTIAL – FOR NeGD(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- DF shows a message:
  "Parental authorization required to proceed."

B) DF Collects Parent/Guardian Email

- DF prompts the minor (or operator in case of POS/branch) to enter:
  - Parent/Guardian Email ID
- DF backend triggers a CMS API call:
  POST /v1/minor-consent/initiate
- CMS sends a secure, time-bound parental consent link to the parent's email.

C) Parent Opens Email Link

- Parent clicks the link → redirected to embedded CMS parental consent journey.
- CMS verifies:
  - Link authenticity
  - Validity window
  - Association with DF request

D) Parent Reviews Consent Template

- The CMS shows a parental consent UI that includes:
- Purpose(s)
- Processing Activities
- Data Items
- Processors
- Sharing scope
- Validity
- Special minor-handling clauses

E) Parent Completes DigiLocker KYC

- Parent is prompted for DigiLocker KYC to verify identity.
- CMS fetches KYC attributes (Name, DOB, masked ID numbers).
- CMS binds KYC data to the minor consent request.

F) Parent Grants Consent

- Parent reviews final consent summary.
- Parent approves.

G) CMS Generates Minor Consent Artifact

- CMS generates:
  - Digitally signed JWS Consent Artifact
  - Includes parent identity, minor reference, and DF metadata
  - Includes a boolean flag: "is_minor_consent": true
- The artifact is stored in immutable storage.

H) Callbacks Sent to DF & Processor

- CMS triggers callbacks to:
- DF
- Data Processor (if any)
- Callback event type:
  MINOR_CONSENT_CREATED
- Each callback includes:
- event_id

"CONFIDENTIAL - FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- minor flag
- timestamp
- signed headers (x-jws-signature)
- link to retrieve full artifact

I) Completion

- DF backend resumes onboarding.
- Parent receives email/SMS confirmation.
- Minor consent appears in the Data Principal dashboard under "Parental Consents" with restricted update options.

4. Architecture Components

The Consent Management System (CMS) is composed of modular microservices and embedded user interface journeys, all operating within a clearly defined architectural boundary. The system enables end-to-end management of personal data consent, cookie preferences, grievances, and user self-service while ensuring security, compliance, and traceability under the DPDP Act.

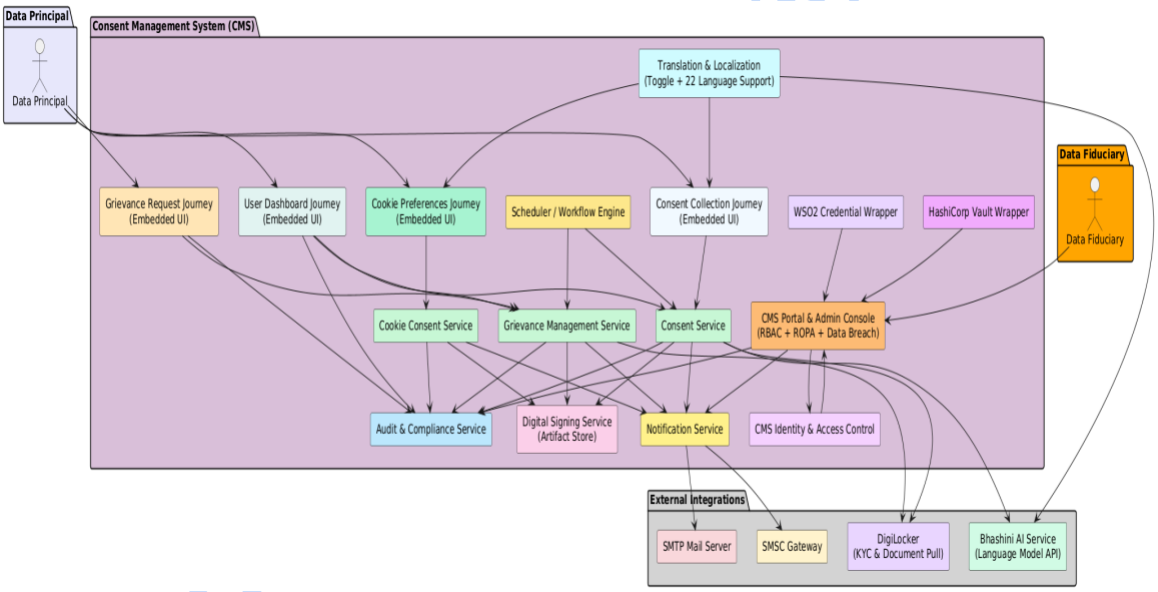

Figure 4: Architectural Components in Consent Management System

The Consent Management System (CMS) architecture consists of four embedded UI journeys, a set of core backend services, and cross-cutting platform services, all designed to deliver a compliant, interoperable, and auditable digital consent ecosystem.

4.1 Embedded UI Journeys (Served by CMS, Embedded by DF)

These journeys run inside the Data Fiduciary (DF) application but are rendered and controlled entirely by CMS, ensuring consistent compliance and UX.

1) Consent Collection Journey

"CONFIDENTIAL - MeitY USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Triggered when a Data Principal must provide consent for data processing.
- Shows the DF-configured consent template, purposes, and processing activities.
- Submits the Data Principal's consent choices to the Consent Management Service.

2) Cookie Preferences Journey

- Displayed during first visit or when a user revisits a DF site/app.
- Enables selection of cookie categories (essential, functional, analytics, marketing).
- Integrates with the Cookie Preference Service for storage and updates.

3) Grievance Request Journey

- Provides a simple interface for filing grievances or rights requests.
- Routes submissions to the Grievance Management Service for ticketing and workflow.

4) User Dashboard Journey

- Allows Data Principals to view prior consents, withdraw or renew them, and track grievance statuses.
- Authenticates the user using CMS Identity & Access Control (email/OTP or bearer validation).
- Interacts with both the Consent Service and Grievance Management Service.

4.2 Core Domain Services

These services implement the core business logic and maintain state for consent, cookie, and grievance management.

1) Consent Management Service

- Handles consent creation, update, renewal, and withdrawal.
- Generates consent artifacts and sends them to the Digital Signing Service.
- Triggers notifications and records all activities into Audit & Compliance logs.

2) Cookie Preference Service

- Stores granular cookie preferences for each Data Principal.
- Maintains versioning and logs every update to ensure audit traceability.

3) Grievance Management Service

- Orchestrates grievance ticket creation, submission, escalation, and closure.
- Integrates with DF callback endpoints and Notification Service.
- Every grievance event is logged and signed before archival.

4.3 Cross-Cutting CMS Platform Services

1) Notification Service

- Handles outbound communication via multiple channels:
- SMSC Gateway for SMS alerts
- SMTP Email Server for email notifications

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- WhatsApp (optional integration)
  Delivers: consent confirmations, updates, renewals, cookie changes, grievance alerts.

2) Digital Signing Service

- Wraps cryptographic signing operations.
- Produces JWS-signed consent and grievance artifacts.
- Works with Artifact Store for immutable archival.

3) Audit & Compliance Service

- Captures transactional events across Consent, Cookie, and Grievance services.
- Uses immutable append-only storage for compliance with DPDP audit requirements.
- Supports compliance reporting and regulatory investigations.

4) CMS Portal & Admin Console

- The administrative console for DF operators & internal CMS operations.
- Supports:
  - Configuration management
  - Legal entity onboarding
  - Role-based access control
  - Master data (Purposes, Processors, Data Items)
  - Consent template creation & publishing
  - Audit log access

5) CMS Identity & Access Control

- Manages authentication for DF Admins and CMS operators.
- Supports email/OTP login and role/rule-based authorization.

6) Scheduler / Workflow Engine

- Handles automated tasks:
- Consent expiry checks
- Re-consent trigger
- Grievance SLA escalations
- Daily audit jobs

7) Translation & Localization Service

- Provides text localization and multilingual support.
- Integrates with Bhashini AI for automated language translations.
- Supports 22 Indian languages for consent UI and notifications.

8) External Integrations

- SMSC (SMS provider) — outbound mobile alerts

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- SMTP (Email provider) — transactional email
- Bhashini AI — multilingual translation & Indian language NLP
- Digilocker — identity/KYC retrieval for special cases (minor consent, strong verification)

4.4 Consent Service

The Consent Service manages the lifecycle of user consent for processing personal data, including collection, updates, withdrawal, and renewal. The Consent Service is structured into modular sub-components orchestrated by the Consent Orchestrator, which acts as the central coordinator for the consent lifecycle.

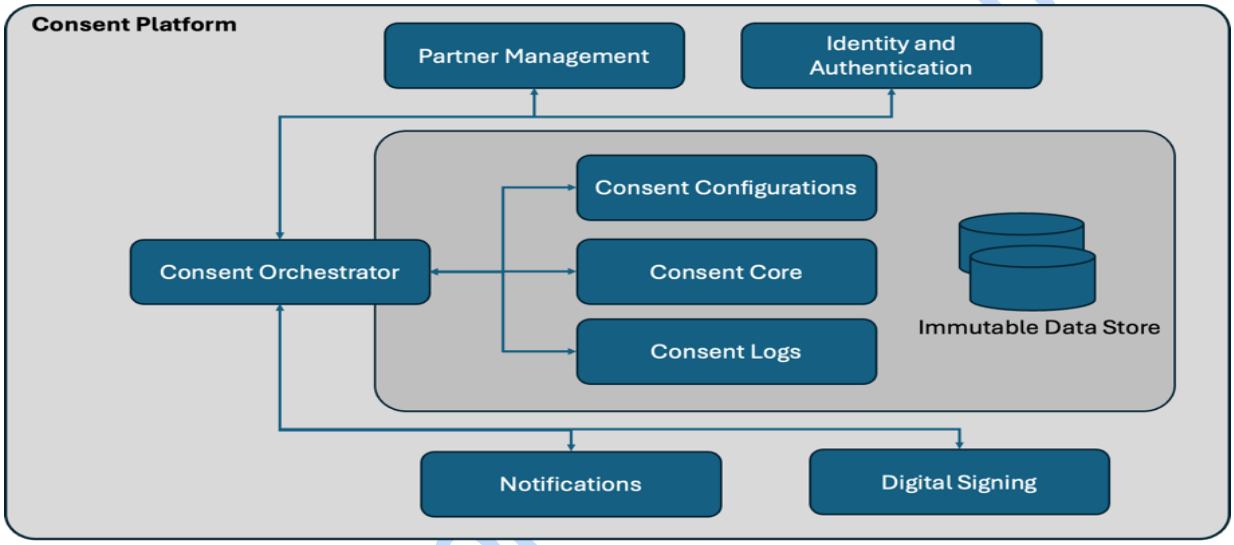

Figure 5: Consent Service Architecture

Its internal architecture includes

1) Consent Orchestrator
   - Central gateway for all consent-related API requests.
   - Coordinates between configurations, core engine, logging, and external services (e.g., notifications, signing).
   - Ensures request validation, delegation, and response handling.

2) Consent Configurations
   - Stores DF-specific rules, policies, and consent template mappings.
   - Supports localization, versioning, and purpose-category tagging.
   - Maintains legal text configurations and UI logic definitions.

3) Consent Core
   - Business logic engine for:
   - Consent grant, update, withdrawal, renewal
   - Status verification
   - Enforces expiry rules, re-consent triggers, and policy application.

"CONFIDENTIAL - FOR USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

4) Consent Logs
- Records all consent-related transactions.
- Pushes logs to Immutable Data Store for tamper-proof compliance storage.
- Integrated with Audit & Grievance services for resolution tracking.

5) Immutable Data Store
- Final resting store for digitally signed, append-only consent records.
- Ensures compliance with non-repudiation and legal audit needs.

6) External Integrations
- Digital Signing for artifact sealing.
- Notifications for user, DF and DP alerting.
- Partner Management and Identity/Auth for DF onboarding and RBAC enforcement.

4.5 Cookie Consent Service

The Cookie Consent Service specifically governs user preferences related to website and application cookies, enabling granular control and ensuring lawful tracking practices. This cookie Consent Service does not provide geographical rules to include as it is expected to work from Indian jurisdiction only.

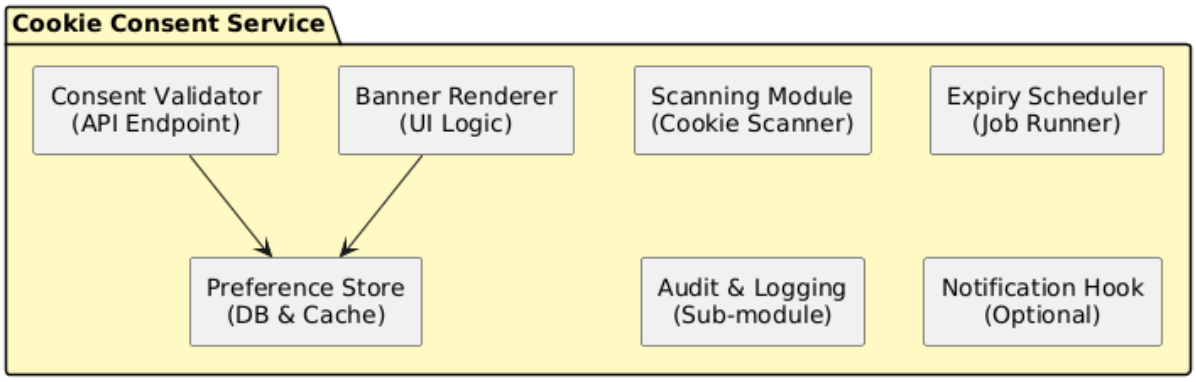

Figure 6: Cookie Consent Management Service

The Cookie Consent Service is responsible for managing user preferences for cookie usage in compliance with data protection regulations. It enables data principals to accept or reject specific categories of cookies (e.g., analytics, advertising) while ensuring auditability and automatic enforcement. The system is composed of the following internal subcomponents:

1) Scanning Module (Cookie Scanner)
Role: Detects all cookies used by a website or platform.
Functionality:
- Automatically identifies new or undocumented cookies from the page/app.
- Maps detected cookies and assigned them to known categories for configuration.

Data Flow:
→ Pushes newly detected cookies into the Preference Store to update available options.
→ Logs scan results to Audit & Logging.

2) Banner Renderer (UI Logic)
Role: Displays a cookie consent banner to the user.
Functionality:

"CONFIDENTIAL - (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Supports category-level preferences (essential, analytics, marketing, etc.).
- Respect localization and accessibility standards.
- Captures user actions (accept all, reject all, customize).

Data Flow:
→ Saves user selections into the Preference Store.
→ Triggered again by Expiry Scheduler if re-consent is needed.

3) Preference Store (DB & Cache)
Role: Stores cookie choices made by users.
Functionality:
- Associates choices with a session ID, browser fingerprint, or user account.
- Stores timestamped decisions for each cookie category.

Data Flow:
← Receives data from Banner Renderer and Scanning Module.
→ Serves current preferences to Consent Validator and Notification Hook.
→ Logs every change to Audit & Logging.

4) Expiry Scheduler (Job Runner)
Role: Automatically enforces re-consent based on configured policies.
Functionality:
- Periodically checks for stale or expired cookie consents.
- Triggers the re-display of the cookie banner when needed.

Data Flow:
→ Reads & updates Preference Store.
→ Informs Banner Renderer to re-trigger UI.
→ Logs expiry events in Audit & Logging.

5) Audit & Logging (Sub-module)
Role: Ensures traceability and accountability.
Functionality:
- Records every change, access, validation, and scan event.
- Streams logs to immutable audit storage for compliance review.

Data Flow:
← Receives events from Scanner, Preference Store, Validator, and Scheduler.

6) Notification Hook (Optional) – Not supported if the User ID is not available.
Role: Informs the user or the DF system about preference updates.
Functionality:
- Sends alerts when cookie settings are modified or expire.
- Supports email, in-app, or webhook-based notifications.
- Data Flow:
  ← Triggered by Preference Store when user actions or expirations occur.

4.6 Grievance Management Service

The Grievance Management Service is a key module of the Consent Management System (CMS) that facilitates the submission, tracking, and resolution of complaints and rights-based requests submitted by data principals. It is designed as a modular, event-driven architecture with embedded UI components on the Data Fiduciary (DF) portal and service orchestration managed entirely within CMS.

"CONFIDENTIAL – FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

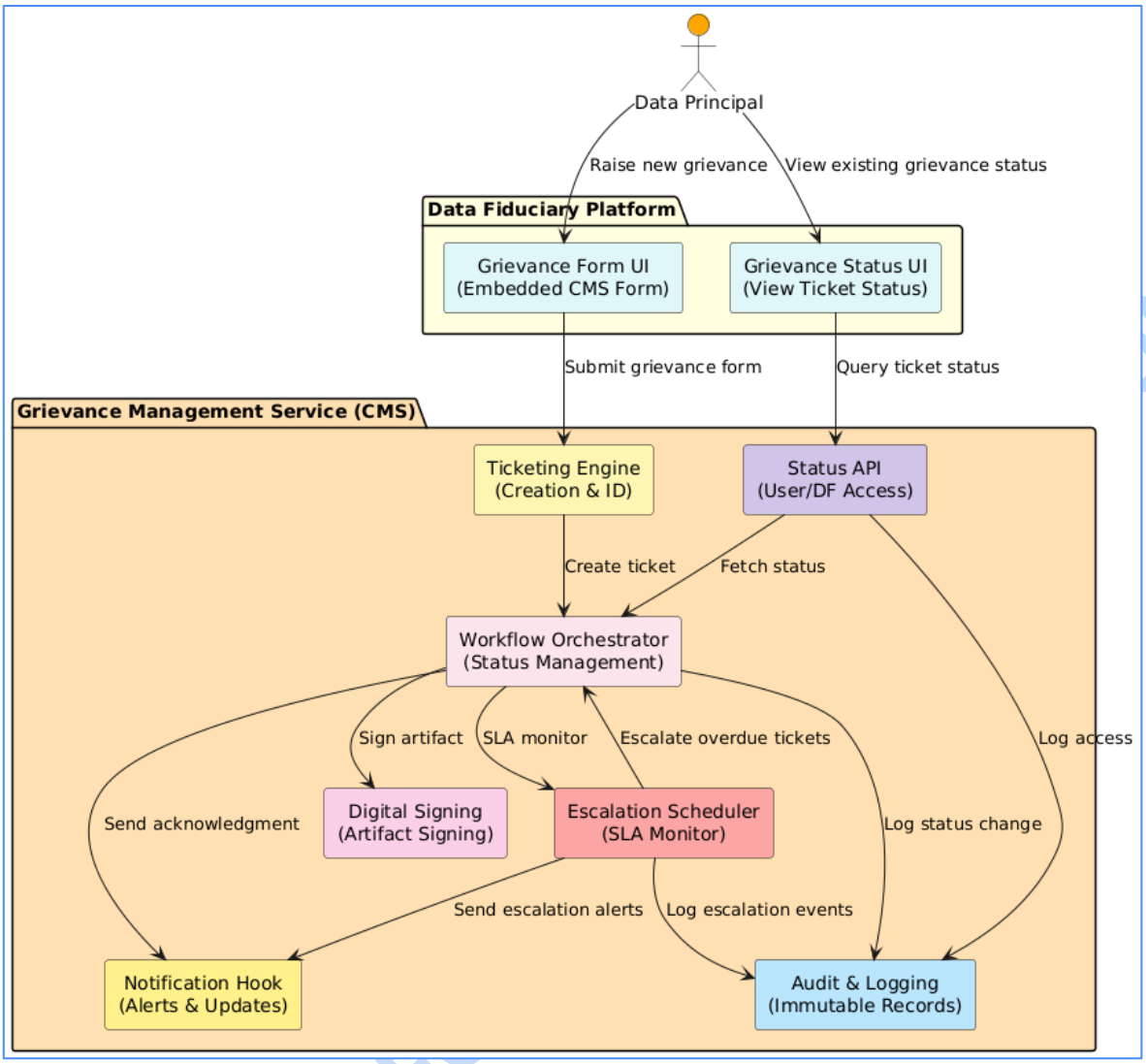

Figure 7: Grievance and Tracking Service Architecture

1) **Data Principal (DP)**: The data principal is the end-user initiating and tracking grievance requests. They interact with the CMS through embedded UI components hosted inside the DF's platform.
2) **Data Fiduciary Platform – Embedded CMS UI Components**
   i. Grievance Form UI (Embedded)
   A pre-built, CMS-hosted form rendered securely within the DF's application.
   Allows the data principal to:
   - Raise complaints
   - Submit data access/correction/erasure/nominate/consent grievance requests
   - Form metadata (fields, validation, labels, consent Ids) are configurable via CMS admin tools.
   - Securely posts data to the CMS backend for ticket creation.

   ii. Grievance Status UI (Embedded)
   Enables the data principal to:
   - View previously raised grievances
   - Check their status (e.g., In Progress, Resolved, Escalated)
   - Authenticated via Identity & Auth layer to ensure access control.

"CONFIDENTIAL - FOR CFC ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Fetches status through the CMS Status API.

3) Grievance Management Service (CMS Subcomponents)

i. Ticketing Engine
- Accepts validated input from the Grievance Form UI.
- Assigns a unique Ticket ID and captures metadata (issue type, timestamps, user).
- Forward the ticket to the Workflow Orchestrator.

ii. Workflow Orchestrator
- Central state manager for grievance tickets.
- Controls transitions across statuses like:
  New → In Progress → Resolved → Closed
- Routes status updates to:
- Notification Hook for communication
- Digital Signing for record sealing
- Audit & Logging for traceability
- Subscribes to escalation triggers via the Escalation Scheduler.

iii. Escalation Scheduler
- Continuously monitors tickets stuck in In Progress or any user-defined state.
- If SLA thresholds are breached:
- Updates the ticket's state to Escalated
- Notifies the responsible administrator or DPO
- Logs the escalation in the audit trail

iv. Status Update API
- Serves read-only, real-time status of grievance tickets.
- Used by:
  - Data Principals via Grievance Status UI
  - Data Fiduciaries via Partner Portal
- Protected by identity and role-based access control.
- Logs every access event for audit purposes.

v. Notification Hook
- Sends alerts and acknowledgments for:
- Ticket submission
- State changes
- Escalation events
- Channels: Email, SMS, in-app, or webhook to DF systems

vi. Digital Signing
- Ensures each grievance submission and resolution step is digitally signed.
- Guarantees authenticity, non-repudiation, and legal defensibility.

vii. Audit & Logging
- Captures:
  - Ticket creation
  - State transitions
  - SLA breaches
  - Status queries
  - Logs are written to an immutable store for audit, compliance, and legal reporting.

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

5. Consent Artifact — Immutability & Crypto Architecture

This section describes the secure, immutable consent storage procedure used by the Consent Management System (CMS). The design guarantees tamper-detection, non-repudiation, long-term auditability, and forensic verification while avoiding storage of plaintext consent payloads.

1) Each business (tenant) has an RSA-4096 keypair (certType = rsa-4096) stored in HashiCorp Vault; Vault returns vaultRef and keyVersion.
2) consentId is a UUID.
3) Canonical JSON, timestamp format, hashing, encoding and signing are deterministic (ISO-8601, sorted keys, consistent encodings).
4) The DB collection model is append-only (Insert / Get only). No updates or deletes.

5.1 System component participated

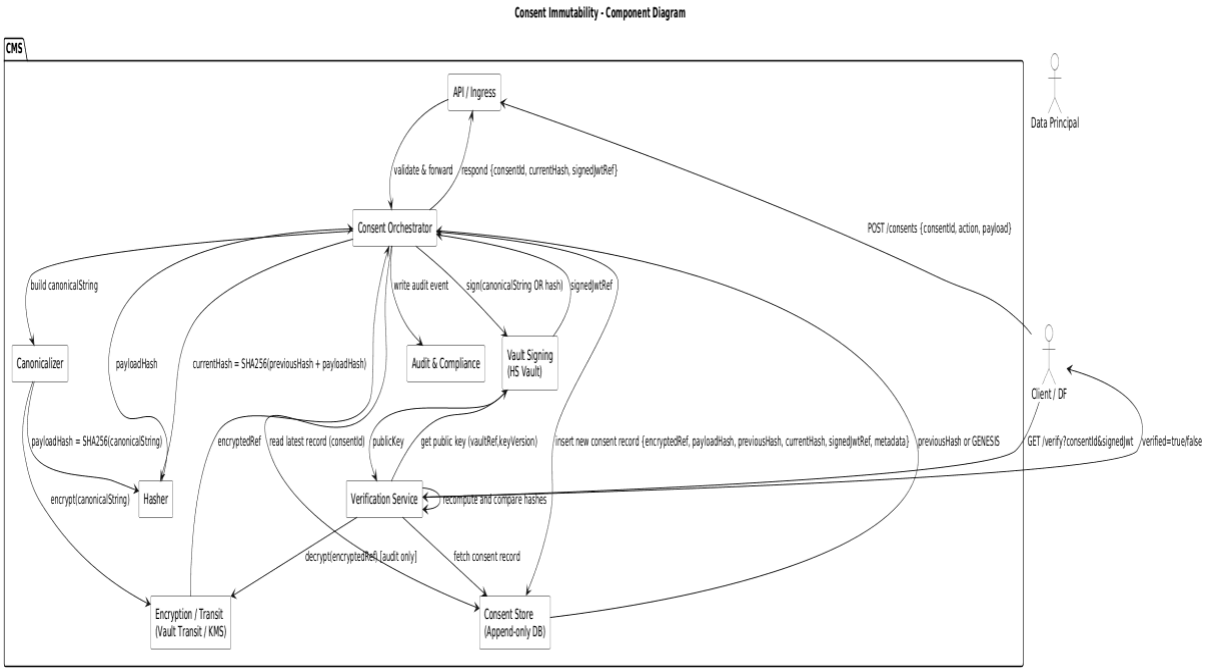

Figure 8: Immutability & Crypto Architecture

1) **API / Ingress —** Validates & authenticates incoming requests (OAuth2, client credentials, DF identity).
2) **Consent Orchestrator —** Business logic, versioning, and high-level workflow.
3) **Canonicalizer** — Produces deterministic canonical JSON string from Consent payload excluding non essential meta data (consentpayload hash, encryptedreferencid, currenthash).
4) **Vault Signing Service** — **Interface** to HashiCorp Vault (signing with RSA-4096).
5) **Encryption Service / Transit** — **Uses** Vault AES256-GCM96 to encrypt canonical JSON and returns an encryptedRef.
6) **Hasher** — **Computes** payloadHash = SHA-256(canonicalString).
7) **Consent Store (append-only DB)** — Stores the consent record document including encryptedRef(vaultRef, keyVersion), payloadHash, currentHash etc.

"CONFIDENTIAL (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

8) **Audit & Compliance** — Stores event logs, supports replay verification.
9) **Verification Service** — Endpoints to validate signed JWT and recompute chain hashes for audit.
10) **Scheduler / DLQ** — Handles retries, atomic persistence, and dead-letter processing.

5.2 Detailed step-by-step procedure (mapping to system components)

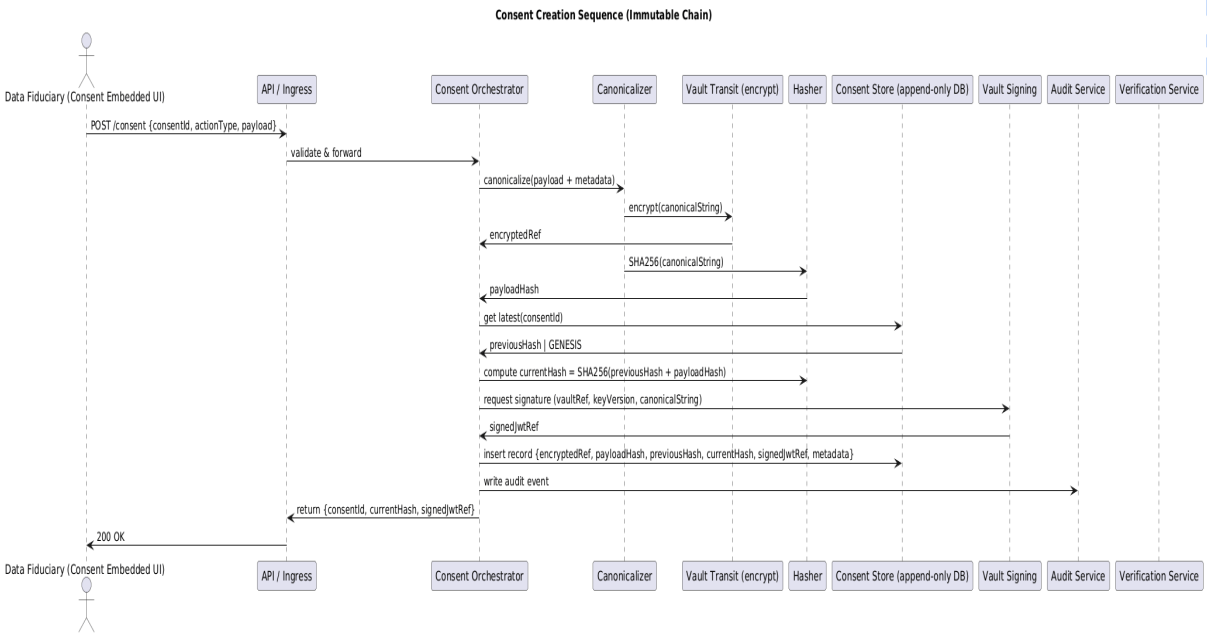

Figure 9: Mapping to system components

Step 1 — Business / Key Onboarding (precondition)

- A RSA-4096 keypair is generated for each legal/business entity and stored in HashiCorp Vault. Vault returns `keyRef`, `keyType`and `publicKey`. The system stores a mapping: `tenantId → businessId → keyRef & publicKey`.

Step 2 — Receive Consent Request

- API receives request: { consentId, actionType, businessId, consentPayload }.
- API applies authentication, authorization, rate limits and schema validation. Rejection returns 4xx.

Step 3 — Build Canonical JSON (Canonicalizer)

- The Canonicalizer normalizes the payload deterministically:
  - Sort object keys,
  - Use ISO-8601 timestamps,
  - Normalize arrays, nulls and whitespace,
  - Remove transient fields (e.g., ephemeral request headers),
  - Output canonicalString.

Step 4 — Request Consent Signature from Vault (Vault Signing Service)

"CONFIDENTIAL - FOR NeGD ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Orchestrator calls Vault signing API with (vaultRef, keyVersion, canonicalString OR hash).
- Vault returns a signed JWT/JWS or signature blob referencing consentId, iat, and keyVersion. Keep signedJwtRef or the detached signature (do not expose secret key).

Step 5 — Encrypt Canonical JSON & Get Encrypted Reference

- Send canonicalString to Vault Transit (or other KMS) for encryption. Receive encryptedRef (an opaque identifier). Plaintext canonicalString is never persisted in DB.

Step 6 — Calculate Payload Hash (Hasher)

- Compute `payloadHash = SHA256(canonicalString)`. Encode consistently `(hex or base64)`.

Step 7 — Fetch Previous Consent Record

- Query consent store for latest record with the same consentId. If found, previousHash = latest.currentHash, else previousHash = GENESIS (system-defined zero-hash).

Step 8 — Compute Current Chain Hash (Hasher)

- Concatenate deterministically: `concat = previousHash + '|' + payloadHash`
- `currentHash = SHA256(concat)`. This chains records to form an immutable audit chain per consentId.

Step 9 — Prepare Consent Record (Record Schema)
Create a DB document (append-only) containing:


```
    "languageSpecificContentMap": {
      "ENGLISH": {
        "description": "Your data will be processed under DPDP legal compliance and used only as per defined purpose.",
        "label": "Consent Required",
        "rightsText": "You may withdraw consent or raise complaints anytime.",
        "permissionText": "By continuing, you are providing lawful processing consent."
      }
    }
  },
  "preferences": [
    {
      "preferenceId": "6cc60479-0448-4aaa-b578-f0cf7bec5f9a",
      "purposeList": [
        {
          "purposeId": "6b9c4395-e8ce-4da6-99b9-407367f313d1",
          "purposeInfo": {
            "purposeId": "6b9c4395-e8ce-4da6-99b9-407367f313d1",
            "purposeCode": "GRIEVANCE_REDRESSAL",
            "purposeName": "Grievance Redressal",
```


"CONSENT FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."


```
            "purposeDescription": "Resolve customer complaints and support tickets effectively.",
            "businessId": "65e3738d-dc47-4f99-8ce5-6075959c4395",
            "scopeType": "TENANT",
            "status": "ACTIVE",
            "createdAt": {
              "$date": "2025-11-18T10:41:04.332Z"
            },
            "updatedAt": {
              "$date": "2025-11-18T10:41:04.332Z"
            }
          }
        }
      ],
      "isMandatory": true,
      "autoRenew": true,
      "preferenceValidity": {
        "value": 12,
        "unit": "MONTHS"
      },
      "startDate": {
        "$date": "2025-11-18T10:41:22.221Z"
      },
      "endDate": {
        "$date": "2026-11-18T10:41:22.221Z"
      },
      "processorActivityList": [
        {
```


Step 10 — Persist Consent Record Atomically

- Only after all crypto steps succeed, insert the document as a new record. If any crypto step fails, abort and return an error. The DB operation is append-only; there are no updates/deletes.

Step 11 — Return Response

- Return minimal references to caller: { consentId, currentHash, signedJwtRef (optional) }. Never return plaintext canonicalString or unencrypted content.

Step 12 — Verification Flow
Verification input: `consentId + signedJwt`. Steps:

1. Fetch consent record(s) for consentId.
2. Verify signedJwt using the public key retrieved from Vault (vaultRef, keyVersion).
3. If signature valid, authenticity & integrity is established.

Step 13 — Audit Verification
For a full audit re-computation:

"CONFIDENTIAL – FOR ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

1. Request the `encryptedRef -> decrypt` via Vault (requires appropriate permissions).
2. Recompute `payloadHash` and `currentHash = SHA256(previousHash + payloadHash)`.
3. Compare recomputed values with stored document—match implies `verified = true`.

5.3 Record & Index design (DB)

- **Collection:** `consents` (one DB per tenant).
- **Indexes**:
  - `{ consentId: 1, createdAt: -1 }` — retrieve latest version quickly.
  - TTL indexes on audit / archival collections per retention policy.

5.4 Security & Operational controls

- **Key management**: All signing keys under HashiCorp Vault, strict ACLs, key rotation via Vault. keyVersion recorded in each record.
- **Encryption**: Canonical JSON encrypted via Vault Transit; only encryptedRef stored.
- **RBAC**: Only authorized system components (service accounts) may call Vault encrypt/sign APIs. DF systems never receive private keys.
- **Immutable storage**: Append-only DB and WORM-style archival for long term retention.
- **Chain Integrity**: SHA-256 chaining ensures tamper-detection; any change breaks chain verification.
- **Atomicity**: If any cryptographic operation fails, the record is not persisted to avoid inconsistent states.
- **Logging**: All orchestration steps written to Audit & Compliance service with correlation IDs.

5.5 Compliance mapping (summary)

- **Immutability** — Append-only DB + chain hash
- **Non-repudiation** — RSA-4096 signatures via Vault
- **Encryption at rest** — Canonical JSON encrypted, only refs stored
- **Auditability** — Chain + signed artifacts + metadata
- **Verification** — Public key verification + recomputation audit

Section 65B custodial certificates may require additional governance steps (legal custodian attestations), however the certificates can be issued from this platform.

6. Integration Interfaces

The Consent Management System (CMS) integrates with external systems and users through well-defined interfaces, enabling seamless operation across Data Fiduciary (DF) platforms, backend systems, and Data Principal (user-facing) journeys. All interfaces are secure, auditable,

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

and designed to enforce consent governance, grievance management, and compliance with data protection laws.

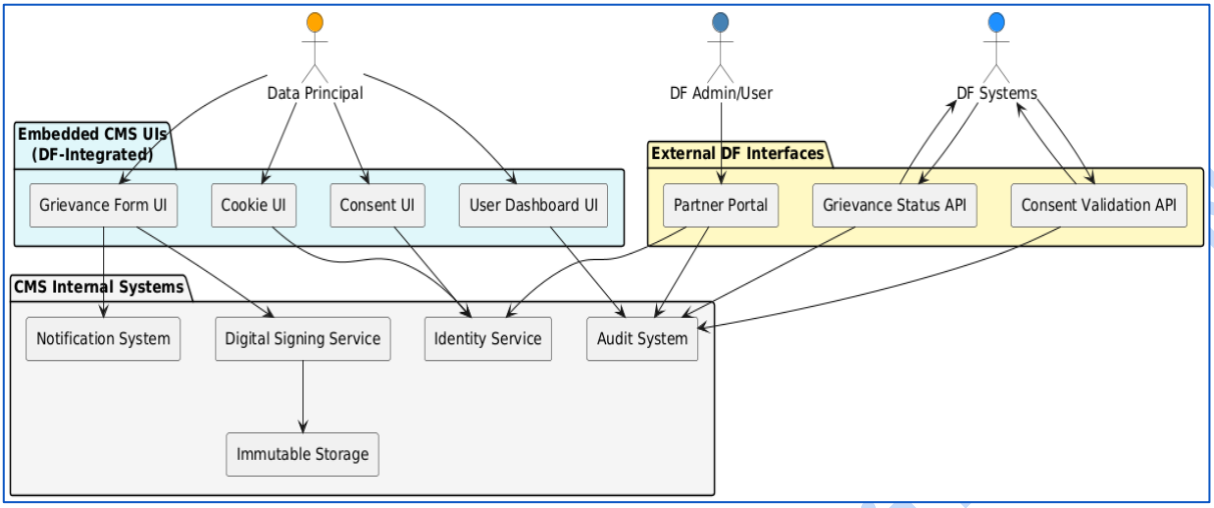

Figure 10: Interfaces Integrations

6.1 Interface Categories and Actors

Interface Category	Description	Actor(s) Involved
Embedded UIs (Client-facing)	UIs hosted by CMS but embedded in DF platforms	Data Principal
External APIs (DF-integrated)	API endpoints used by Data Fiduciary and backends	Data Fiduciary (DF Admin)
System Integrations	Internal services CMS communicates with for signing, auth, etc.	CMS Internal Services

6.2 Embedded User Interfaces (Served by CMS)

These interfaces are embedded within DF apps as iframe, SPA, or redirect components. They are rendered by CMS but integrated natively into the DF environment. All embedded screens are session-authenticated and rendered securely.

Interface	Purpose	Accessed By
Consent Collection UI	Captures data principal consent for personal data processing	Data Principal
Cookie Consent UI	Displays cookie banner for category-specific opt-in preferences	Data Principal
Grievance Form UI	Allows users to file complaints or data rights requests	Data Principal
User Dashboard UI	Lets users view and manage consents, and view grievance ticket status	Data Principal

6.3 System Integrations (Internal Service Interfaces)

These are CMS's internal service interactions with infrastructure and compliance tools.

"CONFIDENTIAL - FOR NeGD/(MeitY) ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Interface	Purpose
Identity & Auth Service	Handles login, session control, and RBAC (for DF admins and users)
Notification System	Sends alerts via email, SMS, or webhook to users and also servers the notifications on the call-back URIs of data fiduciary (DF) and third party data processors
Digital Signing Service	Signs consent, grievance, and status artifacts for non-repudiation
Audit & Logging System	Captures and stores all system and user actions immutably
Immutable Artifact Store	Stores signed artifacts like consent forms and grievances

6.4 Security & Compliance Considerations

1. All interfaces are authenticated via OAuth 2.0/OpenID or signed tokens.
2. Every access to APIs or UI screens is logged via the Audit & Logging System.
3. Sensitive actions (like grievance submission or consent withdrawal) are cryptographically signed.
4. Data is encrypted at rest and in transit, with optional tenant separation for DFs.

6.5 Interface Usage Matrix (Summary)

Actor	Interface	Type	Direction
Data Principal	Consent UI, Cookie UI, Grievance Form, Dashboard	UI	Inbound
DF Systems	Consent Validation API, Grievance Status API, Authentication Service. Call-back listener.	API	Bidirectional
DF Admin/Users	Partner Portal	UI	Inbound
CMS Internal	Identity, Signing, Notification, Audit, Storage	System-to-System	Internal
Processors (Third party)	Consent Validation API, Grievance Status API, Authentication Service. Call-back listener.	API	Bidirectional

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

7. Security Architecture

The Consent Management System (CMS) is designed with a security-first architecture to ensure that all operations involving personal data, consent, grievance handling, and administrative access adhere to privacy regulations and security best practices. The platform enforces robust authentication, granular authorization, end-to-end data protection, and comprehensive audit logging to support non-repudiation, compliance, and incident forensics.

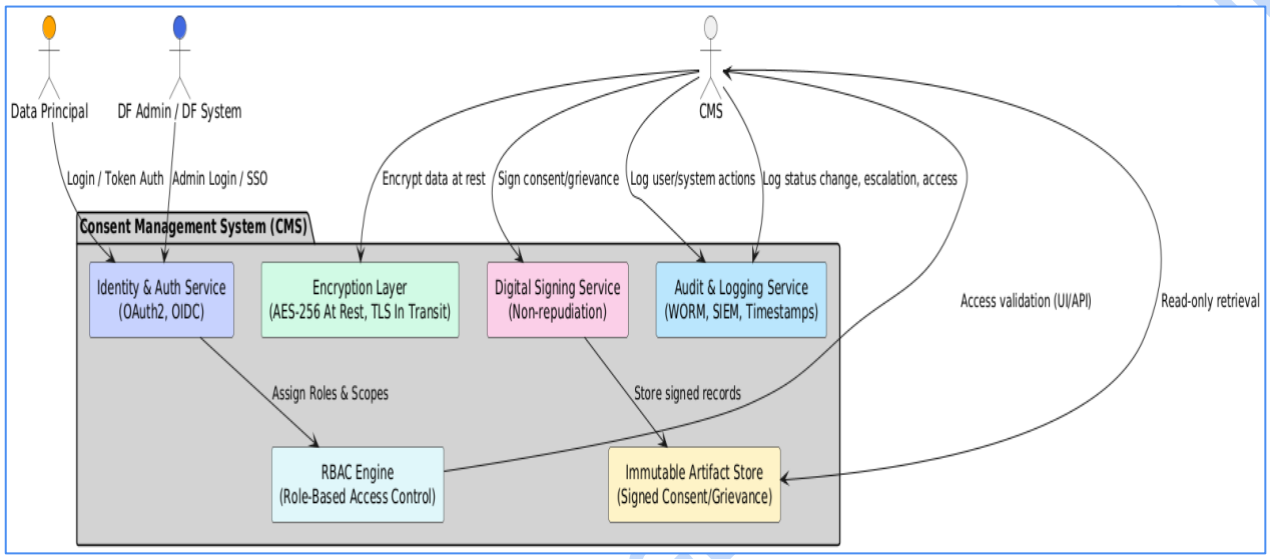

Figure 11: Security Architecture

7.1 Authentication

1) CMS uses a centralized Identity & Authentication Service, supporting OAuth 2.0/OpenID for secure session handling across both UI and API layers.
2) All embedded UIs (e.g., consent, grievance, dashboard) accept signed DF tokens or issue short-lived JWTs upon user session creation.
3) DF Admins authenticate via the Partner Portal, optionally integrated with enterprise SSO (SAML or OIDC).
4) Refresh/access token lifecycle, token introspection, and support for secure headers (Authorization: Bearer) to ensure modern, stateless session control.

7.2 Authorization

1) CMS enforces Role-Based Access Control (RBAC) using identity service claims, scoped tokens, and business-aware role contexts.
2) Roles include DF-admin, DPO, and CMS-admin, mapped via onboarding metadata and maintained in the Partner Management module.
3) All API endpoints and UI modules validate the role and scope at runtime.
4) Admin-level permissions (e.g., managing grievance categories or consents) are enforced through policy filters inside the orchestration services.

"CONFIDENTIAL – FOR NeGD ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

7.3 Data Protection

1) Transport security: All data-in-motion is encrypted via TLS 1.3.
2) At-rest encryption: Databases and log stores use AES-256, with optional field-level encryption for PII fields (personal data fields).
3) Digital Signing: All consent and grievance artifacts are cryptographically signed using CMS's Digital Signing Service, with hashes stored in the Immutable Artifact Store.
4) Retention and purge policies are applied per DF configuration; grievances and consents can be auto-purged, archived, or anonymized post-resolution using scheduled jobs and data lifecycle controllers.

7.4 Audit & Logging

1) A centralized Audit & Logging Service captures all sensitive operations (e.g., login, consent actions, grievance transitions).
2) Logs are immutable, timestamped, and include metadata such as user ID, IP address, and event type.
3) Stored in a WORM-compliant storage layer (Write Once Read Many), integrated with the Immutable Store.
4) Supports real-time export to SIEM platforms via webhook or streaming sinks (e.g., Kafka, ELK, or Splunk integrations).

7.5 Security in Notifications

Layer	Enforcement
Call-back Ping	HMAC (X-CMS-Signature) or JWT (Authorization header)
Pull API	OAuth2 / JWT-based token authorization
Transport	HTTPS with TLS 1.3 enforced
Rate Limiting	Applied per DF/DP registration

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

8. Data Architecture

The Data Architecture of the Consent Management System (CMS) is designed to ensure structured, secure, and compliant handling of data across all services. It supports modular service boundaries, tenant-level isolation, secure storage practices, and lifecycle-aware retention, enabling traceability, performance, and regulatory alignment with frameworks such as the DPDP Act and ISO 27001.

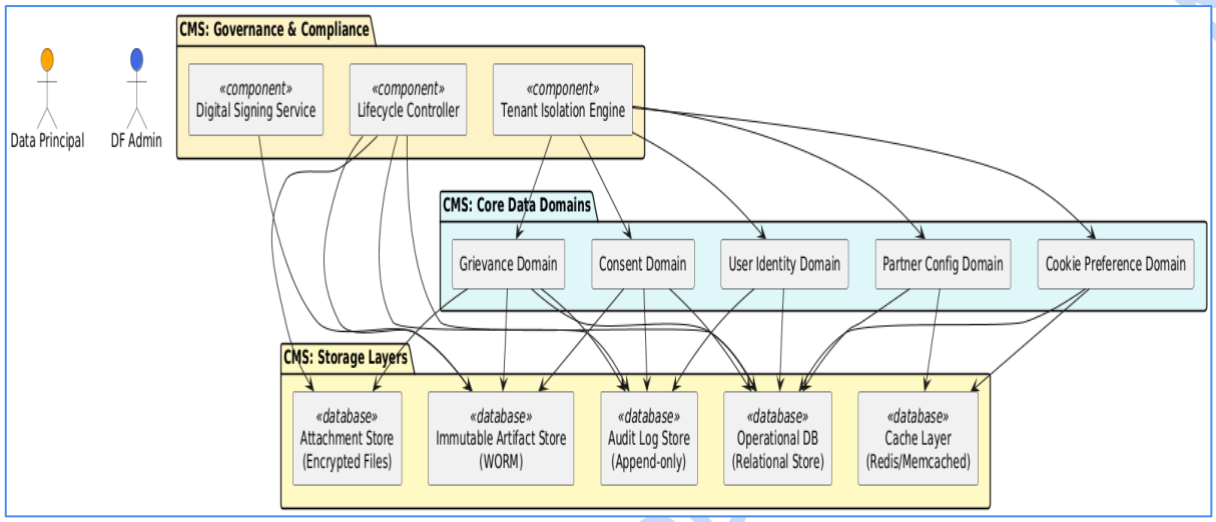

Figure 12: Data Architecture

8.1 Domain-Oriented Design

CMS adopts a modular, domain-driven data architecture. Each functional area maintains its own isolated data models and service boundaries. Each domain owns its schema, versioning, signing logic, and API surface — no cross-service DB access is permitted.

Domain	Ownership Responsibility
Consent	Lifecycle of data processing permissions
Cookie Preferences	UI decisions and session-level consent categories
Grievance	Ticketing, resolution, SLA compliance
User Identity	Authentication tokens and DF-linked identities
Partner Configuration	DF-specific policies, onboarding, and branding

8.2 Storage Strategy & Principles

CMS applies a layered hybrid storage model optimized for security, auditability, and access control. All storage is encrypted at rest and in transit. Sensitive fields follow optional field-level encryption or hashing.

Storage Tier	Purpose
Relational DB	Active operational data (consents, Cookie Consents, grievances, etc.)

"CONFIDENTIAL - FOR NeGD ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Immutable Artifact Store	WORM storage for signed records
Audit Log Store	Append-only logs of all sensitive user/system events
Cache Layer	Accelerated reads for configs, preferences
File Store (Optional)	Secure storage for grievance attachments

8.3 Data Retention and Lifecycle Policies

CMS enforces configurable data lifecycle rules:

1) Consents: Retained for a defined duration post-withdrawal; then archived or purged.
2) Grievances: Retained based on regulatory SLA (typically 5–7 years).
3) Cookies: Automatically expire on TTL or browser reset.
4) Audit Logs: Maintained for a minimum of 7 years and stored immutably.

Each Data Fiduciary can customize their own retention and archival policies, supporting erasure requests and legal holds.

8.4 Multi-Tenant Isolation

All data is tenant-aware and isolated at DB level. Each Data Fiduciary operates in a multi-tenant model:

1) Tenant identifiers are embedded in every data entity.
2) Access is scoped by tenant ID.
3) Signing and storage layers are isolated for each tenant to apply DF-specific cryptographic keys where applicable.

8.5 Governance and Compliance

CMS is designed to comply with data protection regulations (DPDP) through:

1) Digital signing of all consent and grievance artifacts
2) Immutable audit trails for every user or system action
3) Right to access/erase workflows handled through grievance automation
4) Tamper-evident storage of all sensitive records

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

9. Deployment Architecture

The Consent Management System (CMS) is designed for high availability, regulatory reliability, and multi-tenant scalability. The deployment architecture supports secure, modular microservice deployment with auto-scaling, stateless orchestration, and resilience across all critical components.

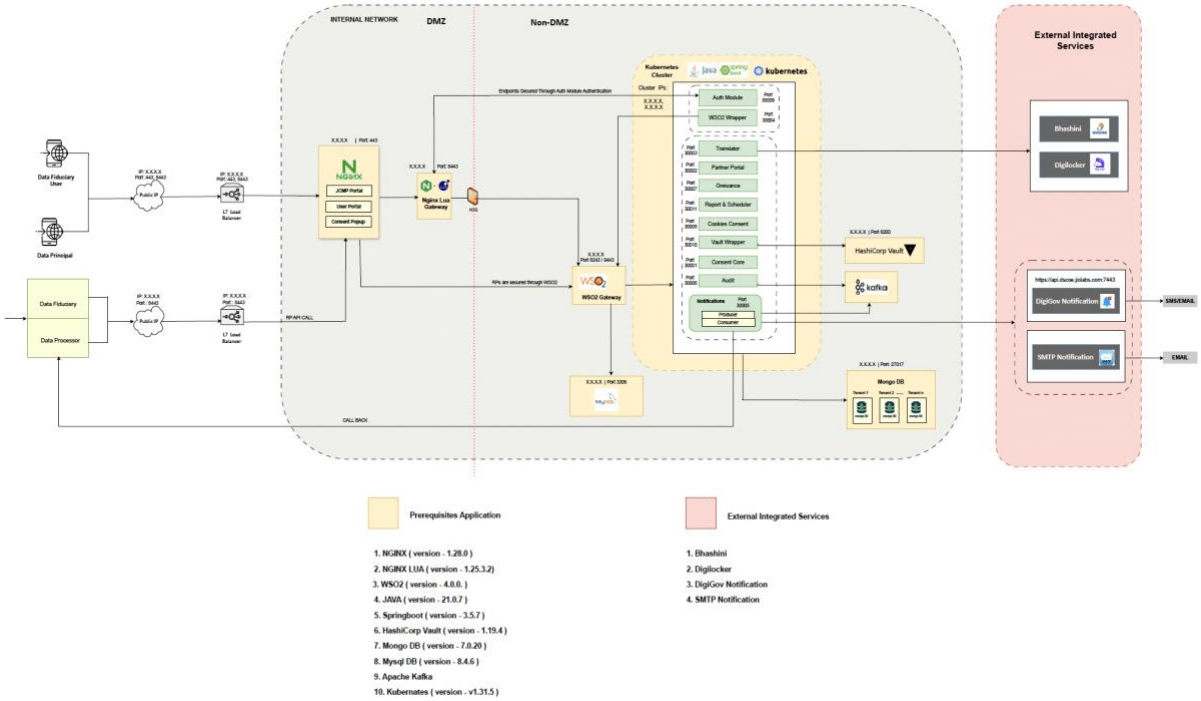

Figure 13: Deployment Diagram

9.1 Deployment Environments

CMS supports multi-environment deployment to separate dev/test workloads from production-grade regulatory operations. Each environment is isolated with independent infrastructure, tenant mapping, and signing credentials.

Environment	Purpose	Key Features
Dev/Stage	Internal testing, staging for DFs	CI/CD-driven, isolated sandbox
UAT/Pre-Prod	Partner testing and pre-launch validation	Near-prod parity, access controlled
Production	Live, regulatory-grade platform	Multi-zone, secure, monitored

9.2 Containerized Microservice Deployment

1) CMS components (Consent Core, Grievance, UI Gateways, Auth, Notification, etc.) are deployed as Docker containers.
2) Orchestrated via Kubernetes (K8s) or managed alternatives like Azure or AWS.

"CONFIDENTIAL – FOR ... ILY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

3) Stateless services use horizontal pods; persistent layers (DBs, audit logs, object stores) use managed storage services.
4) CI/CD Pipelines: Azure DevOps-based pipelines enforce policy validation, code scanning, and stage-specific rollouts.

9.3 Scaling Strategy

CMS is built with a scale-out first strategy to accommodate peak data principal traffic, grievance surges, or consent validation loads. Auto-scaling policies based on CPU, memory, and custom metrics (e.g., queue depth, SLA latency).

Component	Scaling Mode	Triggers
Consent APIs	Horizontal (stateless)	Incoming TPS/load
Grievance Orchestrator	Worker pool	Ticket queue length
Notification Dispatcher	Event-driven	Notification batch size
UI Services	Stateless scaling	Concurrent sessions
DBs/Stores	Vertically scaled	Managed services (IO thresholds)

9.4 Failover & High Availability

CMS is engineered for zero data loss and minimal downtime, with the following resilience patterns:
Multi-Zone Redundancy: Services and data replicated across availability zones

Capability	Implementation Details
Multi-Zone Resilience	Services deployed across availability zones
Health Monitoring	Readiness and liveness probes + open source monitoring tech stack
Disaster Recovery	Snapshots + cross-region backup with <15m RTO
Traffic Control	Load balancers with active-passive or weighted routing
Retry Queues	Built-in DLQs for workflows like escalation, notifications

9.5 Blue-Green Deployment Strategy

CMS uses a blue-green deployment model in production to ensure zero-downtime updates and safe rollback capabilities. The Blue environment is the live production environment currently serving user traffic. A new version of the application is deployed to the Green environment. Once validation and smoke tests pass, the load balancer is switched from Blue to Green. In case of failure, rollback is immediate by redirecting traffic back to Blue.

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

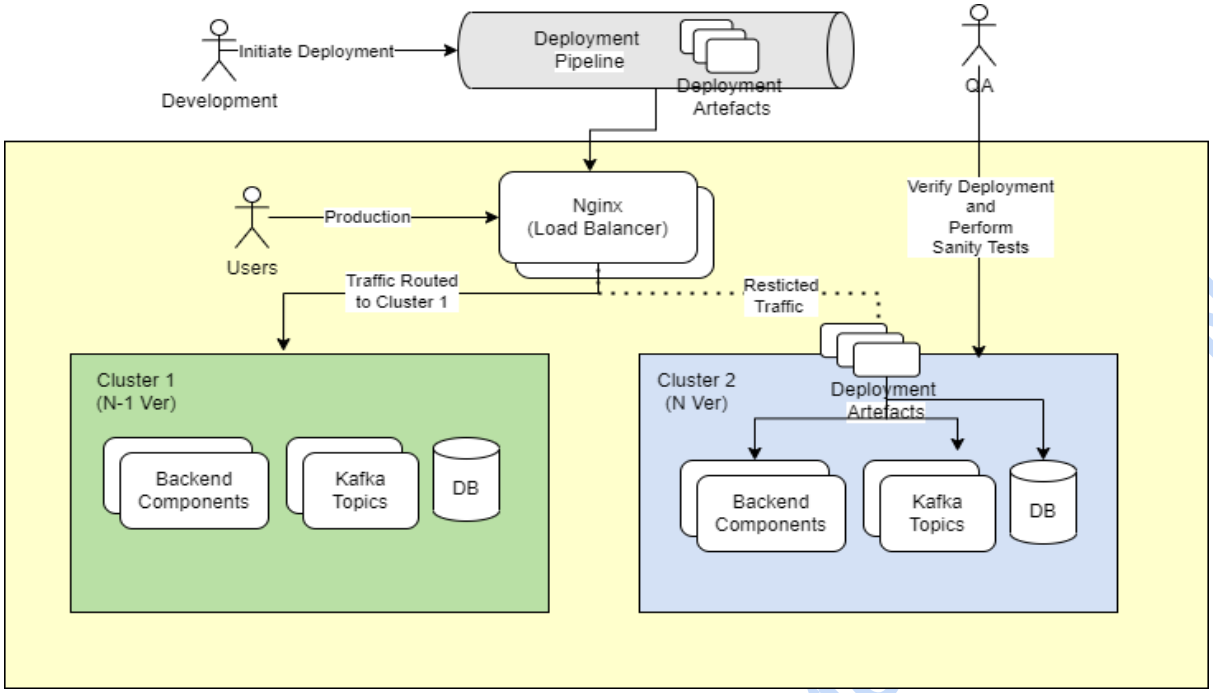

Figure 14: Blue Green Deployment Architecture

"CONFIDENTIAL – FOR NeGD(MeitY) ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

10. Compliance & Governance

The Consent Management System (CMS) is built to ensure strict alignment with India's Digital Personal Data Protection (DPDP) Act, while also addressing general principles from ISO 27701. The architecture, policies, and operational practices embedded in the platform are designed to enforce data minimization, consent traceability, individual rights enforcement, and privacy by design.

1) Consent Lifecycle Management (Section 6, 7 of the DPDP Act)
   - CMS enforces granular, purpose-linked, revocable consent with full traceability.
   - Consent artifacts are digitally signed, time-stamped, and versioned.
   - Users can withdraw consent at any time via the self-service UI.
   - DFs can only access data post having a valid active consent.
   - Consent updates (renewal, purpose change) require re-acceptance.
   - Mapped to: DPDP Section 6(3), 6(4), 6(5)

2) User Rights & Grievance Handling (Section 11)
   - Embedded Grievance Form UI lets users raise complaints or exercise rights (access, correction, erasure).
   - SLA-driven ticket orchestration ensures timely resolution.
   - Escalation workflows automatically notify DPOs or authorities if unresolved.
   - Users can track the status of their grievances transparently.
   - Mapped to: DPDP Section 11(2), 11(3), 11(4)

3) Data Principal Dashboard
   - A unified, embedded dashboard provides:
     - History of given consent
     - Withdrawal or update options
     - Status of grievance requests
     - Enables full visibility and control to the data principal.
   - Mapped to: DPDP Right to Access and Correction (Section 11)

4) Digital Signing & Audit Trail
   - All consent and grievance actions are digitally signed to ensure non-repudiation.
   - Immutable logs are maintained for:
     - Consent given/withdrawn
     - Data accessed
     - Tickets raised/resolved
     - WORM (Write Once Read Many) store ensures audit logs are tamper-evident.
   - Mapped to: DPDP Section 7(3), Compliance Logging Requirements

5) Retention & Erasure
   - Data retention periods are configurable per DF and purpose.
   - Lifecycle controller:
     - Archives expired data
     - Supports erasure requests post-verification
     - Grievances can trigger secure deletion flows with signing & logging.
   - Mapped to: DPDP Section 8(4), 8(6), 8(8)

"CONFIDENTIAL – FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

6) Purpose, Limitation & Minimization
   - Each consent is linked to one or more defined purposes.
   - Consent attributes are limited to only what is declared.
   - DFs must define purpose templates and data types before requesting consent.
   - Mapped to: DPDP Section 5(1), 6(2)
7) Multi-Tenant Isolation & Role-Based Access
   - Data is scoped by tenant (DF) using tenant IDs at every level.
   - Access is governed by the RBAC engine, with roles such as: df-admin, df-dpo, cms-admin.
   - Logs capture who accessed what and when.
   - Mapped to: DPDP Section 9(3), 9(4)

8) Security & Privacy Controls
   - Platform enforces encryption at rest and in transit.
   - Tokens and sessions use OAuth 2.0 / OIDC.
   - Consent banners and UIs are dynamically localized and customized per DF to ensure privacy by design.
   - Mapped to: DPDP Section 8(5), 8(7)

9) Multilingual Support
   - Consent and grievance UIs are rendered in DF-defined local languages
   - Language fallback and right-to-left (RTL) layout support
   - Useful for jurisdictions with linguistic diversity and local compliance mandates
   - Mapped to: DPDP Section 6 (informed consent, accessibility)

10) Accessibility (WCAG 2.1 Compliance)
   - CMS UIs follow WCAG 2.1 Level AA guidelines
   - Includes keyboard navigation, screen reader compatibility, and high-contrast mode
   - Ensures all users, including those with disabilities, can interact with the platform
   - Mapped to: DPDP Section 11 (Inclusive Access & Transparency)

"CONFIDENTIAL – FOR NeGD/MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix A: Glossary

Term	Definition
Consent Management System	The core platform responsible for collecting, managing, validating, and storing user consent and grievances.
Data Principal	An individual to whom the personal data relates; the end user of the system under the DPDP Act.
Data Fiduciary (DF)	An entity (organization or partner) that determines the purpose and means of processing personal data.
Data Processor (DP)	A third-party service or system that processes personal data on behalf of a Data Fiduciary.
Consent Artifact	A digitally signed and versioned record of a user's consent, linked to a purpose and data attributes.
Purpose	The legal or business justification for collecting and processing personal data, as declared to the user.
Grievance Ticket	A formal request raised by a data principal to access, correct, erase, or complain about data misuse.
Grievance Escalation	Workflow triggered when a grievance is not resolved within the SLA or requires DPO intervention.
Immutable Artifact Store	A WORM (Write Once, Read Many) compliant storage service that holds digitally signed consents and grievances.
Digital Signing Service	A CMS service that applies cryptographic signatures to artifacts to ensure non-repudiation.
RBAC (Role-Based Access Control)	Authorization mechanism that restricts system access based on roles like data-principal, df-admin, etc.
OI/OIDC (OpenID Connect)	A secure authentication protocol built on OAuth2, used for federated login and token issuance.
OAuth2	A protocol for delegated authorization used to issue access and refresh tokens for UI/API access.
Lifecycle Controller	Service responsible for managing the retention, archival, and deletion of data per DF-defined policies.
Audit Log Store	An append-only, tamper-evident logging service that captures all user and system actions.
WORM (Write Once Read Many)	A data storage model that prevents alteration of stored records after initial write, ensuring immutability.
Partner Portal	A web-based admin console of CMS platform for DF onboarding, RBAC configuration, consent template setup, and monitoring.
User Dashboard UI	Embedded UI that allows users to view and manage their consents and grievance ticket statuses.
Consent Validation API	An interface that enables DF systems to check if valid consent exists before initiating data processing.
Grievance Form UI	Embedded module allowing users to submit new grievances through the DF portal.

"CONFIDENTIAL – FOR NeGD/MeitY USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Blue-Green Deployment	A zero-downtime deployment model using two identical environments (Blue and Green) for safe release cycles.
Tenant ID (df_tenant_id)	A unique identifier used to logically isolate each DF's data and configurations in a multi-tenant setup.
WCAG 2.1	Web Content Accessibility Guidelines — CMS UIs are designed to conform to AA level accessibility standards.
Multilingual Support	The ability to localize all UI elements (consent forms, grievance screens, banners) into multiple languages.
Consent Banner	A UI component displayed to collect user preferences for cookies and trackers.
Cookie Preferences	User-defined opt-in/out settings for various cookie categories (e.g., analytics, marketing).
Retention Policy	Rules defining how long each data type (consent, grievance, logs) is stored before archiving/deletion.
Right to Erasure	A user's right under DPDP to request deletion of their personal data, facilitated through grievance flows.
SIEM (Security Information and Event Management)	A monitoring system used for analyzing and responding to logs and alerts from the CMS.
DPDP Act	India's Digital Personal Data Protection Act – governing data protection, consent, access, and penalties.
Privacy by Design	Architectural principle that embeds privacy features and safeguards into system design and operations.

"CONFIDENTIAL – FOR NeGD/MeitY USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix B: JWS Signed Consent Artifact – Technical Specification

Consent artifacts are cryptographically signed representations of the consent granted by a data principal to a data fiduciary, issued and managed by the Consent Management System (CMS). To ensure integrity and authenticity, the artifact is packaged as a JWS (JSON Web Signature) object. This section provides an overview of its structure, usage, and technical components.

Structure:

```
consentJwtToken:

Header:
{

"alg"
: "RS256",

"typ"
: "JWT"
}
```

1) Purpose and Security Rationale

- Tamper-Proof Representation: Signing consent artifacts with a private key ensures no post-issuance modifications.
- Detached and Embedded Signature Mechanism: The artifact is protected by a dual signature approach. A detached signature is returned in the x-jws-signature header, while an embedded signed_consent token is included within the payload. Both are signed using the CCA-compliant private key.
- Decentralized Verification: Any authorized consumer (like DF, DP, or audit systems) can verify the payload using CMS's public key.
- Data Governance Compliance: Digitally signed artifacts are aligned with DPDP and privacy legislation for traceability and legal admissibility.
- Detached Signature: The actual payload is transferred as a JSON body, and the signature is shared via x-jws-signature HTTP header, enabling secure validation without bloating the response.

2) Structure of Consent Artifact (JWS Payload)

he payload is a structured JSON containing metadata, user identity, processing details, and sharing permissions.

Field	Description
timestamp	ISO-8601 time of consent issuance
consent_artifact_id	Unique identifier for tracking and audit
version	Version of the consent form/template
language_preference	Language used in the UI presented to user
data_principal	Pseudonymized user identifier (e.g. phone/email hash)
data_fiduciary	Organization receiving the consent (ID + name)
consent_details	List of purposes, attributes, sharing and duration

3) Sample Consent Payload

```
Body:
{
  "sub": "{\"consentId\":\"816f41e0-6e26-4e02-ae8f-
bdb76aa55da6\",\"consentHandleId\":\"0beb6b43-e389-47ec-ab7a-
```

"CONFIDENTIAL - FOR NeGD(MeitY) ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."


```
0778228c46df\",\"templateId\":\"23ed23c0-04f1-480c-ac5f-
e11d2541fa6d\",\"templateName\":\"Test1\",\"templateVersion\":1,\"businessId\":\"d7fab84a-
fbcb-4350-8eb3-
356abfa9065a\",\"languagePreferences\":\"ENGLISH\",\"multilingual\":{\"supportedLanguages\"
:[\"ENGLISH\"],\"languageSpecificContentMap\":{\"ENGLISH\":{\"description\":\"WhileusingJio
Meet,youractivitiescreatedatawhichwillbeusedwithyourconsenttooffercustomisedservices.Detail
sofdatausageareprovidedbelow.\",\"label\":\"Required\",\"rightsText\":\"Towithdrawyourconse
nt,exerciseyourrights,orfilecomplaintswiththeBoardclickhere.\",\"permissionText\":\"Byclick
ing'Allowall'or'Savemychoices',youareprovidingyourconsenttoRelianceMedlabusingyourdataasout
linedabove.\"}}},\"preferences\":[{\"preferenceId\":\"d31b380c-5809-4d0a-b7ba-
3da39af4ae83\",\"purposeList\":[{\"purposeId\":\"5d7b13c4-6df8-4563-a7d7-
3359b23734cf\",\"purposeInfo\":{\"purposeId\":\"5d7b13c4-6df8-4563-a7d7-
3359b23734cf\",\"purposeCode\":\"P001\",\"purposeName\":\"AccountCreation&Authentication\",
\"purposeDescription\":\"Collectdatatocreateuseraccountandauthenticatesecurely.\",\"busines
sId\":\"d7fab84a-fbcb-4350-8eb3-
356abfa9065a\",\"scopeType\":\"TENANT\",\"status\":\"ACTIVE\",\"createdAt\":\"2025-11-
19T19:09:23.356\",\"updatedAt\":\"2025-11-
19T19:09:23.356\"}}],\"isMandatory\":true,\"autoRenew\":true,\"preferenceValidity\":{\"valu
e\":1,\"unit\":\"YEARS\"},\"startDate\":\"2025-11-21T05:39:20.636\",\"endDate\":\"2026-11-
21T05:39:20.636\",\"processorActivityList\":[{\"processorActivityId\":\"d3f9ae89-ba78-4e2d-
b0f5-0bbd2b107984\",\"processActivityInfo\":{\"processorActivityId\":\"d3f9ae89-ba78-4e2d-
b0f5-
0bbd2b107984\",\"activityName\":\"JIOMEETTEST\",\"processorName\":\"JIOMEET\",\"processorId
\":\"3049e7da-b5a4-4d07-a2b0-e7bbdfbf98fc\",\"dataTypesList\":[{\"dataTypeId\":\"b2c3d4e5-
f678-9012-3456-
7890abcdef12\",\"dataTypeName\":\"ContactData\",\"dataItems\":[\"PhoneNumber(Mobile/Landlin
e)\"]}],\"details\":\"eadc\",\"businessId\":\"d7fab84a-fbcb-4350-8eb3-
356abfa9065a\",\"scopeType\":\"TENANT\",\"status\":\"ACTIVE\",\"version\":1,\"createdAt\":\
"2025-11-19T19:15:27.104\",\"updatedAt\":\"2025-11-
19T19:15:27.104\"}}],\"preferenceStatus\":\"ACCEPTED\"}],\"customerIdentifiers\":{\"type\":
\"EMAIL\",\"value\":\"MAYUR.JALORA@RIL.COM\"},\"startDate\":\"2025-11-
21T05:39:20.636\",\"endDate\":\"2026-11-21T05:39:20.636\",\"status\":\"ACTIVE\"}",
  "iss": "JIO CONSENT",
  "exp": 1795239560,
  "iat": 1763703560
}

Signature:
```


4) Signature & Headers

- Detached JWS Signature: The CMS signs the consent artifact metadata using its private key (issued by CCA). This JWS signature is returned in the x-jws-signature header.
- Embedded Signed Consent in Payload: In addition to the detached signature, the payload also contains a signed_consent field which is a JWS token signed with the same CCA private key. This embedded token offers a tamper-evident representation of the user's original consent.

"CONFIDENTIAL - FOR NeGD/MeitY USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix C: Authentication – Identifying the Data Principal

The Data Fiduciary (DF) must authenticate the data principal before presenting any consent or grievance UI. This is implemented using:

5) Auth 2.0 Flow

1) The DF integrates CMS via OAuth 2.0 Authorization Code Flow.
2) The data principal authenticates using DF's login mechanism (e.g., phone OTP, username/password).
3) On success, the DF issues an access token and ID token (OIDC) which is:
   - Passed to the embedded CMS UI for session bootstrapping
   - Used to link all actions (consent, grievance) to a verified identity

6) OpenID Connect (OIDC) Integration

1) CMS accepts standard OIDC ID tokens for identity resolution.
2) User profile claims (e.g., sub, phone_number, email) are extracted to link the action to the authenticated data principal.
3) If the DF is using an identity provider (e.g., Auth0, Keycloak, Aadhaar stack), the same OIDC token is accepted by CMS.
4) It provides following advantages:
   - No duplicate authentication
   - Single sign-on (SSO) across CMS and DF systems
   - Trust relationship established via DF-configured client credentials

7) Identity in JWT Consent Artifact

The field:

```
"data_principal": {
  "id": "+918433980719",
  "type": "Phone"
}
```

Is populated using the identifier verified via OIDC. It may be hashed, encrypted, or pseudonymized depending on the DF's risk posture.

8) Operational Flow Summary

Step	Action
DF Authenticates User	OAuth2/OIDC login → generates tokens
CMS UI Uses Token	Embedded CMS UI receives identity → links user to data principal context
Consent Captured & Signed	CMS signs and returns JWT consent artifact
DF Stores Artifact	DF stores token or uses in data processing validation calls
DP or DF Validates Artifact	Validates signature + expiry + scope before accessing personal data

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Appendix D: Notifications Service Architecture

The architecture leverages a dynamic collection strategy that isolates logs per event type, preventing data conflicts and making inspection straightforward for developers and testers. Its path-based routing mechanism supports multiple event flows with a minimal controller footprint, while the fully config-driven design eliminates dependencies on external systems, allowing seamless and reproducible testing environments. The audit-centric data model ensures end-to-end traceability of each notification, and the plug-and-play forwarding layer accurately mirrors production behavior for realistic validation. Looking ahead, enhancing path matching with pattern-based routing, supporting multiple forwarding configurations, improving null-safe JSON parsing, and adopting a reactive WebClient for scalable forwarding can further strengthen reliability and performance of the service.

The Notification Service provides following features:

- System-to-System Call-backs to DFs and DPs for integration automation.
- User Notifications via SMSC and SMTP for transparency and compliance.
- Resilience mechanisms such as retries, audit logging, and missed-event retrieval APIs.

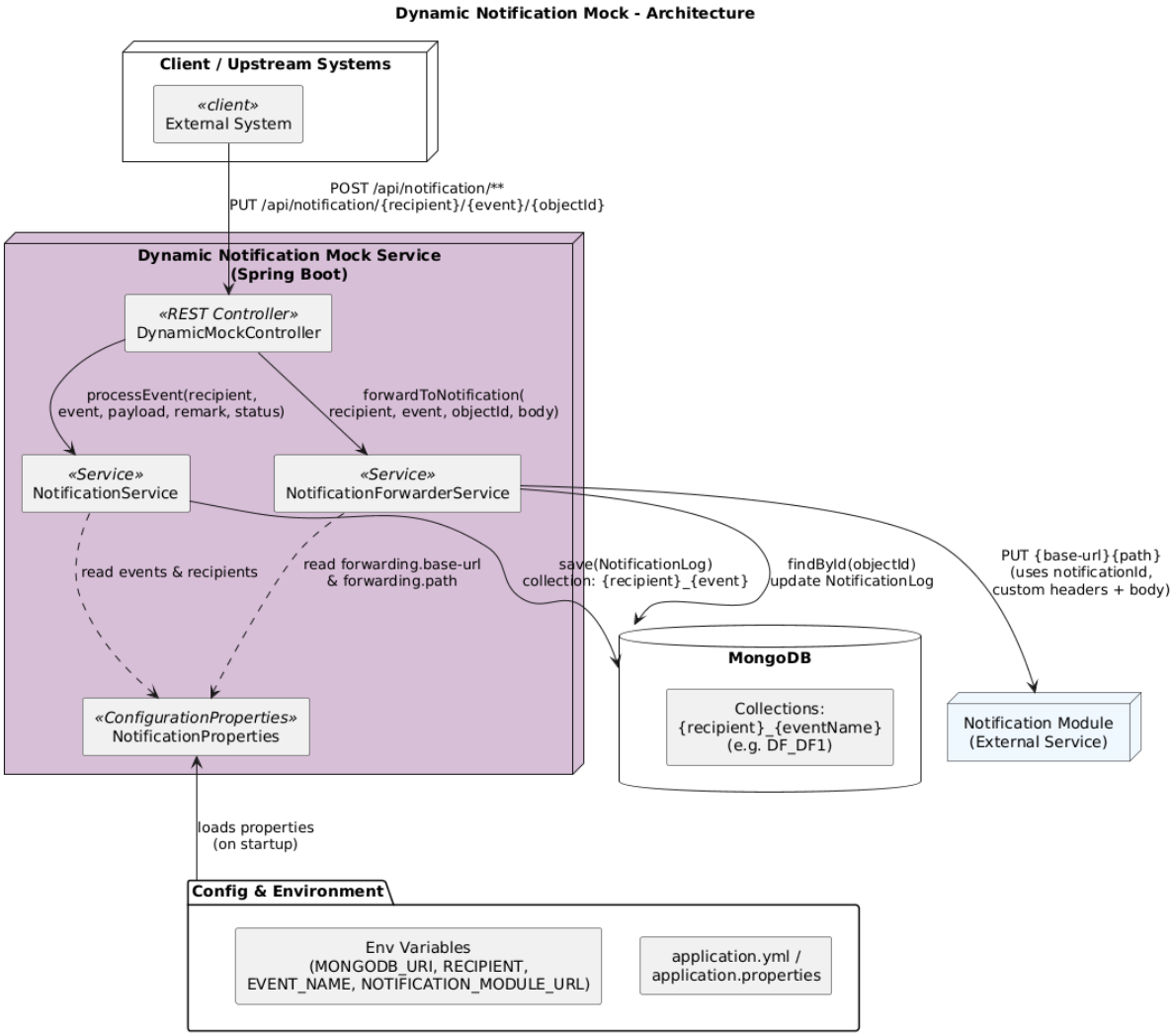

Figure 15: CMS Notification Architecture

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

The Dynamic Notification Service follows a modular, service-oriented architecture built on Spring Boot, designed to simulate the entire lifecycle of notification callbacks. Incoming REST requests are handled by the `DynamicMockController`, which routes POST callback events and PUT forwarding requests to the appropriate internal services. The behaviour of the mock is completely configuration-driven through `NotificationProperties`, enabling dynamic mapping of callback paths, recipients, event names, and forwarding destinations without code changes. This configuration layer abstracts event handling logic away from controller code, allowing the application to scale easily when new events or recipients are added. The design leverages Spring Boot's auto-binding, environment variables, and hierarchical YAML configurations for flexible deployment across local, QA, and integration test environments.

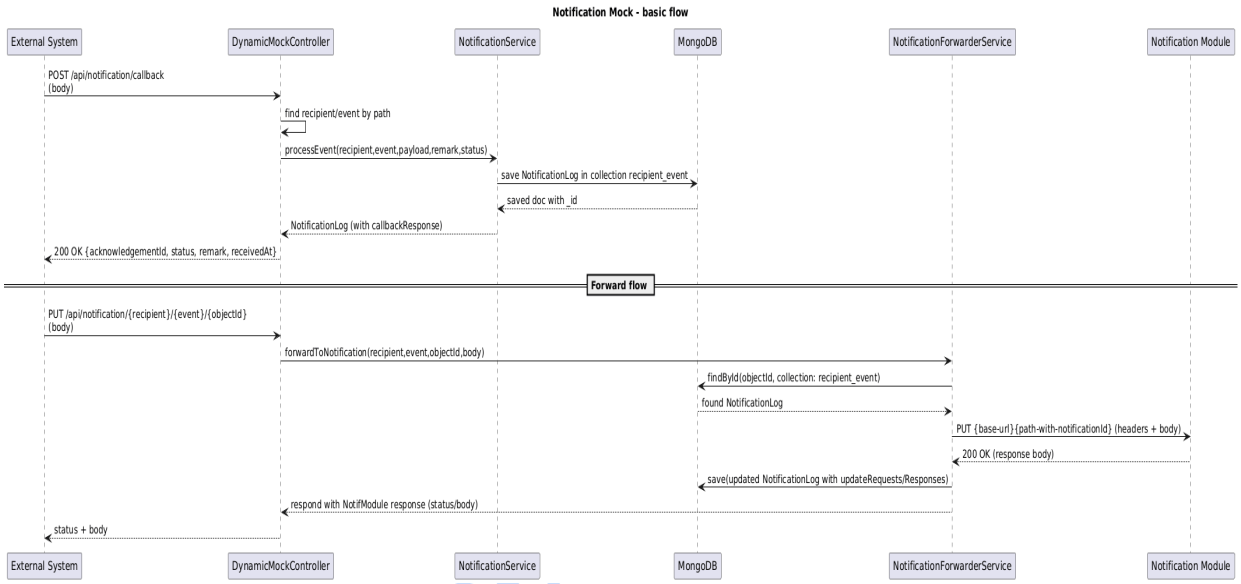

Figure 16: CMS Notification sequence diagram (Internal)

A key architectural component is the use of MongoDB as a dynamic persistence layer, where callback data is stored in collections named per recipient-event (e.g., `DF_DF1`, `DF_DP1`). This structure offers clean isolation of logs, simplifies debugging, and allows testers to easily inspect event histories. The `NotificationService` stores callback requests and acknowledgment payloads, while the `NotificationForwarderService` enriches the stored records with forwarding requests and responses, acting as an intermediary between the mock and the real Notification Module. By centralizing all interactions in a single `NotificationLog` document, the system ensures full traceability of the request lifecycle. The forwarding pipeline dynamically builds headers and URLs using stored payload metadata, offering a realistic simulation of production behaviour while maintaining a clean separation between controller, business logic, configuration, and data storage layers.

High-Level Component

1) DynamicMockController

- `POST /api/notification/**` - Single catch-all endpoint for all callback events using @PostMapping("/**"). - Iterates through configured recipients and events to find matching path. - Logs received event to console with recipient and payload

"CONFIDENTIAL – FOR CFC ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

details. - Returns acknowledgment with acknowledgementId, status, remark, receivedAt.
  - Maps request path → configured recipient + event.
- `PUT /api/notification/{recipientName}/{eventName}/{objectId}**` - Loads stored callback record and forwards status updates to the Notification Module. - Path variables: recipientName (DF/DP), eventName (DF1/DP1), objectId (MongoDB _id). - Request body contains status and remark for the status `update.NotificationService`

2) NotificationService

- Processes incoming callback events via processEvent(recipient, eventType, payload, remark, status):

  1) Creates new NotificationLog instance
  2) Sets eventType from matched event configuration
  3) Stores original payload in callbackRequest
  4) Generates callbackResponse containing:
  5) acknowledgmentId - `UUID` generated via UUID.randomUUID().toString()
  6) status - From YAML configuration (e.g., "ACKNOWLEDGED")
  7) remark - From YAML configuration
  8) receivedAt - Current timestamp via new Date().toInstant()
  9) Saves to dynamic collection using mongoTemplate.save(log, collectionName)
  10) Collection name pattern: {recipient}_{eventType} (e.g., DF_DF1)

3) NotificationForwarderService

- Forwards stored notifications to Notification Module via forwardToNotification(recipientName, eventName, objectId, body):
- Validates input - Checks `recipientName`, `eventName`, `objectId` are not blank
- Loads record - mongoTemplate.findById(objectId, NotificationLog.class, collectionName)
- Validates mandatory fields in `callbackRequest`:
  - recipientType
  - recipientId
  - tenantId
  - businessId
  - Retrieves configuration - Finds `RecipientConfig` and `ForwardingConfig` from `NotificationProperties`
- Builds URL dynamically: url = forwarding.getBaseUrl() + forwarding.getPath() .replace("{notificationId}", payload.get("notificationId").toString())
- Builds HTTP headers via buildHeaders() :
  - Content-Type: application/json
  - RequestedType - From stored payload
  - Recipient-Id – From stored payload
  - X-Business-Id – From stored payload
  - X-Tenant-Id – From stored payload
  - Additional custom headers from `ForwardingConfig`

"CONFIDENTIAL - FOR NeGD/MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Sends `PUT` request - Using `RestTemplate` to forward to Notification Module
- Updates record:
  - Appends request body to updateRequests[]
  - Appends parsed response to updateResponses[]
  - Saves updated record via mongoTemplate.save(record, collectionName)

4) MongoDB Storage Layer

- Dynamic collections per recipient-event: -
  Pattern: `{recipient}_{eventType}` -
  Examples: `DF_DF1, DF_DF2, DP_DP1, DP_DP2`
- Each collection stores `NotificationLog` documents using `MongoTemplate` for dynamic collection naming:
  - `String collectionName = recipient + "_" + eventType;`
  - `mongoTemplate.save(log, collectionName);`

5) Configuration Architecture

Configuration is loaded via Spring Boot @ConfigurationProperties, supporting YAML or properties format.

Key config structures and design characteristics

```
server:
    port: ${SERVER_PORT:9011}

   spring:
     data:
       mongodb:
         uri: ${MONGODB_URI:mongodb://localhost:27017}
         database: ${MONGODB_DATABASE:dpdp_callback}

   notifications:
     recipients:
       - recipient: ${RECIPIENT:DF}
         events:
           - name: ${EVENT_NAME:DF1}
             path: /api/notification/callback
             remark: "Notification Event acknowledged"
             status: "ACKNOWLEDGED"
         forwarding:
           - base-url: ${NOTIFICATION_MODULE_URL:http://localhost:9003}
             path: /notification/v1/missed-
   notifications/{notificationId}/status
             headers:
               X-Custom-Header: "custom-value"
```

Configuration Classes

Class | Purpose | | -------- | -------- | | NotificationProperties | Root config with List<RecipientConfig> recipients | | RecipientConfig |

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Contains recipient, List<EventConfig> events, List<ForwardingConfig> forwarding | | EventConfig | Contains name, path, remark, status | | ForwardingConfig | Contains baseUrl, path, Map<String, String> headers |

- Path-based mapping: POST URL matched via event.getPath().equalsIgnoreCase(requestPath).
- Dynamic forwarding URL: {notificationId} placeholder replaced at runtime from stored payload.
- Environment-driven setup: MongoDB URI, event names, recipients, and base URLs are externalized for flexible deployments.
- Multi-instance support: Multiple instances can run simultaneously for different recipients (DF on xxxx, DP on xxxx).

6) Data Model (NotificationLog)

A single MongoDB document represents the full lifecycle of a notification. The design ensures auditable traceability of all callback flows.

```
    @Data
    @Document
    public class NotificationLog {
        @Id
        private String id;
        private String eventType;
        private Map<String, Object> callbackRequest;
        private Map<String, Object> callbackResponse;
        private List<Map<String, Object>> updateRequests;
        private List<Map<String, Object>> updateResponses;
}
```

Field	Type	Purpose
id	String	MongoDB document identifier (_id)
eventType	String	Event identifier (DF1, DP1, etc.)
callbackRequest	Map<String, Object>	Original incoming JSON payload from Consumer
callbackResponse	Map<String, Object>	Acknowledgement returned to Consumer
updateRequests	List<Map<String, Object>>	Every forwarded PUT request body to Notification Module
updateResponses	List<Map<String, Object>>	Parsed response from Notification Module

callbackResponse Structure

Field	Description
acknowledgmentId	UUID generated for tracking
status	Status from configuration (e.g., "ACKNOWLEDGED")
remark	Remark from configuration
receivedAt	Timestamp when callback was received (Instant)

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Expected `callbackRequest` Fields (from Consumer)

Field	Description
`notificationId`	Unique notification identifier from Consumer (required for forwarding)
`recipientType`	DATA_FIDUCIARY or DATA_PROCESSOR (mandatory for forwarding)
`recipientId`	Target recipient identifier (mandatory for forwarding)
`tenantId`	Tenant identifier for multi-tenancy (mandatory for forwarding)
`businessId`	Business unit identifier (mandatory for forwarding)
`data`	Event-specific payload data

7) Runtime Flow Architecture

a) Callback (POST) Sequence

- Notification Consumer → Controller (`POST /api/notification/callback`)
- Controller gets full request path via `request.getRequestURI()`
- Controller iterates through `notificationProperties.getRecipients()` to find matching path: `java recipient.getEvents().stream() .filter(e -> e.getPath().equalsIgnoreCase(path)) .findFirst();`
- If no match found, returns `400 Bad Request` with "Unknown path" message
- Controller calls `NotificationService.processEvent(recipient, eventName, payload, remark, status)`
- NotificationService:
  - Creates `NotificationLog` with incoming payload
  - Generates UUID for `acknowledgmentId`
  - Sets configured `status` and `remark`
  - Saves to `{recipient}_{eventType}` collection
  - Returns saved NotificationLog
  - Controller extracts response fields and returns acknowledgment: `json { "acknowledgementId": "550e8400-e29b-41d4-a716-446655440000", "status": "ACKNOWLEDGED", "remark": "Notification Event acknowledged", "receivedAt": "2025-01-20T10:30:00.000Z" }`

b) Forwarding (PUT) Sequence

- External System → Controller (`PUT /api/notification/{recipientName}/{eventName}/{objectId}`)
- Controller calls `NotificationForwarderService.forwardToNotification(recipientName, eventName, objectId, body)`
- `ForwarderService` validates input parameters are not blank
- Constructs collection name: `recipientName + "_" + eventName`
- Loads `NotificationLog` by `objectId` from collection
- Validates mandatory fields in `callbackRequest`:
- Returns 400 Bad Request if `recipientType`, `recipientId`, `tenantId`, or `businessId` is missing

"CONFIDENTIAL - FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

- Retrieves `RecipientConfig` and `ForwardingConfig` from `NotificationProperties`
- Builds forwarding URL: `http://localhost:9003/notification/v1/missed-notifications/{notificationId}/status`
- Builds `HTTP` headers with X-`Recipient-Type`, X-`Recipient-Id`, X-`Business-Id`, `X-Tenant-Id`
- Initializes `updateRequests` and `updateResponses` lists if null
- Appends current request body to `updateRequests[]`
- Sends `PUT` via `RestTemplate`
- Parses response and appends to `updateResponses[]`
- Saves updated record to MongoDB
- Returns response from Notification Module

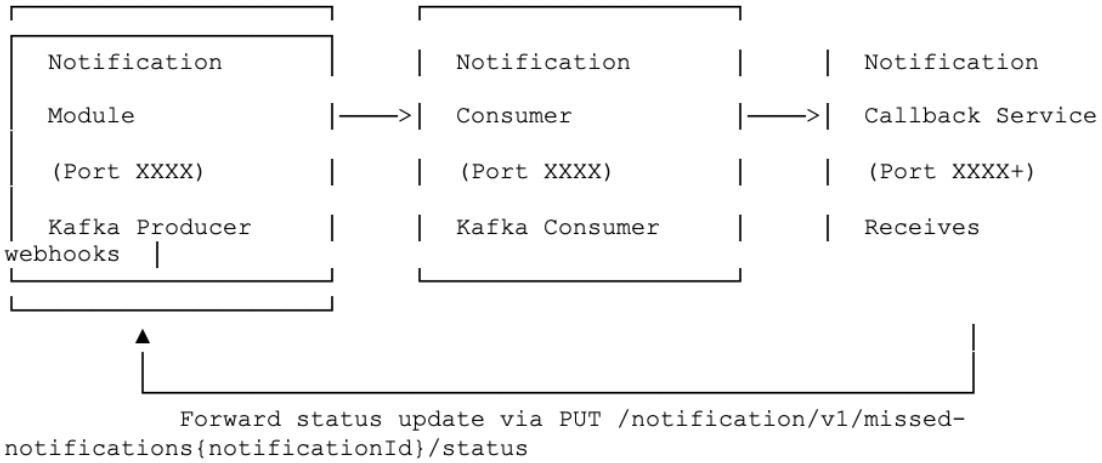

Figure 17: Integration with Notification System

The Callback Service acts as the endpoint for Data Fiduciaries (DF) and Data Processors (DP), receiving webhook notifications from the Consumer and optionally forwarding status updates back to the Module for complete lifecycle tracking.

Event Type	Notify Data Fiduciary	Notify Data Processor	Notify Data Principal	Notify DPO	Notify CMS	Priority	Description
CONSENT_REQUEST_PENDING	FALSE	FALSE	TRUE	FALSE	FALSE	MEDIUM	Notification when consent request is pending
CONSENT_RENEWAL_REQUEST	FALSE	FALSE	TRUE	FALSE	FALSE	MEDIUM	Notification when consent renewal is requested
CONSENT_CREATED	TRUE	TRUE	TRUE	FALSE	FALSE	HIGH	Notification when consent is created
CONSENT_UPDATED	TRUE	TRUE	TRUE	FALSE	FALSE	HIGH	Notification when consent is updated
CONSENT_WITHDRAWN	TRUE	TRUE	TRUE	FALSE	FALSE	HIGH	Notification when consent is withdrawn

"CONFIDENTIAL - FOR ... USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

CONSENT_EXPIRED	TRUE	TRUE	TRUE	FALSE	FALSE	HIGH	Notification when consent has expired
CONSENT_RENEWED	TRUE	TRUE	TRUE	FALSE	FALSE	HIGH	Notification when consent is renewed
GRIEVANCE_RAISED	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is raised
GRIEVANCE_INPROCESS	TRUE	FALSE	TRUE	TRUE	FALSE	MEDIUM	Notification when grievance is in process
GRIEVANCE_ESCALATED	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is escalated
GRIEVANCE_RESOLVED	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is resolved
GRIEVANCE_DENIED	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is denied
GRIEVANCE_CLOSED	TRUE	FALSE	TRUE	TRUE	FALSE	MEDIUM	Notification when grievance is closed
GRIEVANCE_STATUS_UPDATED	FALSE	FALSE	TRUE	TRUE	TRUE	MEDIUM	Notification when grievance status is updated
DATA_DELETED	TRUE	FALSE	TRUE	TRUE	TRUE	HIGH	Notification when data is deleted
DATA_SHARED	TRUE	FALSE	TRUE	TRUE	TRUE	HIGH	Notification when data is shared
DATA_BREACHED	FALSE	FALSE	TRUE	FALSE	FALSE	HIGH	Notification when data breach is detected
DATA_RETENTION_DURATION_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when data retention duration expires
LOG_RETENTION_DURATION_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	MEDIUM	Notification when log retention duration expires
TENANT_ONBOARDED	FALSE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when new tenant is onboarded to the system
INIT_OTP	FALSE	FALSE	TRUE	FALSE	FALSE	HIGH	Notification to send OTP for authentication or verification
CONSENT_PREFERENCE_EXPIRY	FALSE	TRUE	TRUE	FALSE	FALSE	MEDIUM	Notification when consent preference is due to expire
TOTP_PIN_DETAILS	FALSE	FALSE	TRUE	FALSE	FALSE	HIGH	Notification to send TOTP setup details for 2FA
ORGANIZATION_ONBOARDING	FALSE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when new organization is onboarded

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

PROCESSOR_ONBOARDING	FALSE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when new data processor is onboarded
DPO_USER_ONBOARDING	FALSE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when new DPO user account is created
GRIEVANCE_ESCALATED_L1	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is escalated to Level 1
GRIEVANCE_ESCALATED_L2	TRUE	FALSE	TRUE	TRUE	FALSE	HIGH	Notification when grievance is escalated to Level 2
DATA_EXPIRED	TRUE	FALSE	TRUE	TRUE	TRUE	HIGH	Notification when data has expired
GRIEVANCE_RETENTION_DURATION_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when grievance retention duration expires
CONSENT_RETENTION_DURATION_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when consent retention duration expires
RETENTION_CONSENT_ARTIFACT_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when consent artifact retention period expires
RETENTION_COOKIE_CONSENT_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when cookie consent retention period expires
RETENTION_GRIEVANCE_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when grievance retention period expires
RETENTION_LOGS_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when system logs retention period expires
RETENTION_DATA_EXPIRED	FALSE	FALSE	FALSE	TRUE	FALSE	HIGH	Notification when data retention period expires
CONSENT_INTEGRITY_VERIFICATION_FAILED	FALSE	FALSE	FALSE	TRUE	FALSE	CRITICAL	Critical alert when consent integrity verification fails
CONSENT_INTEGRITY_VERIFICATION_COMPLETED	FALSE	FALSE	FALSE	TRUE	FALSE	MEDIUM	Notification when consent integrity verification completes successfully
COOKIE_CONSENT_HANDLE_CREATED	TRUE	FALSE	FALSE	FALSE	FALSE	MEDIUM	Notification when cookie consent handle is created

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

COOKIE_CONSENT_CREATED	TRUE	FALSE	FALSE	FALSE	FALSE	HIGH	Notification when cookie consent is created
NEW_COOKIE_CONSENT_VERSION_CREATED	TRUE	FALSE	FALSE	FALSE	FALSE	HIGH	Notification when new cookie consent version is created
TOKEN_VALIDATION_SUCCESS	TRUE	FALSE	FALSE	FALSE	FALSE	MEDIUM	Notification when token validation succeeds
COOKIE_CONSENT_REVOKED	TRUE	FALSE	FALSE	FALSE	FALSE	HIGH	Notification when cookie consent is revoked
COOKIE_CONSENT_EXPIRED	TRUE	FALSE	FALSE	FALSE	FALSE	HIGH	Notification when cookie consent is expired

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

End of Document

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"